

Agent项目-基于LLM的金融分析AI Agent

1. 项目概述（公众号【辅论AI】）

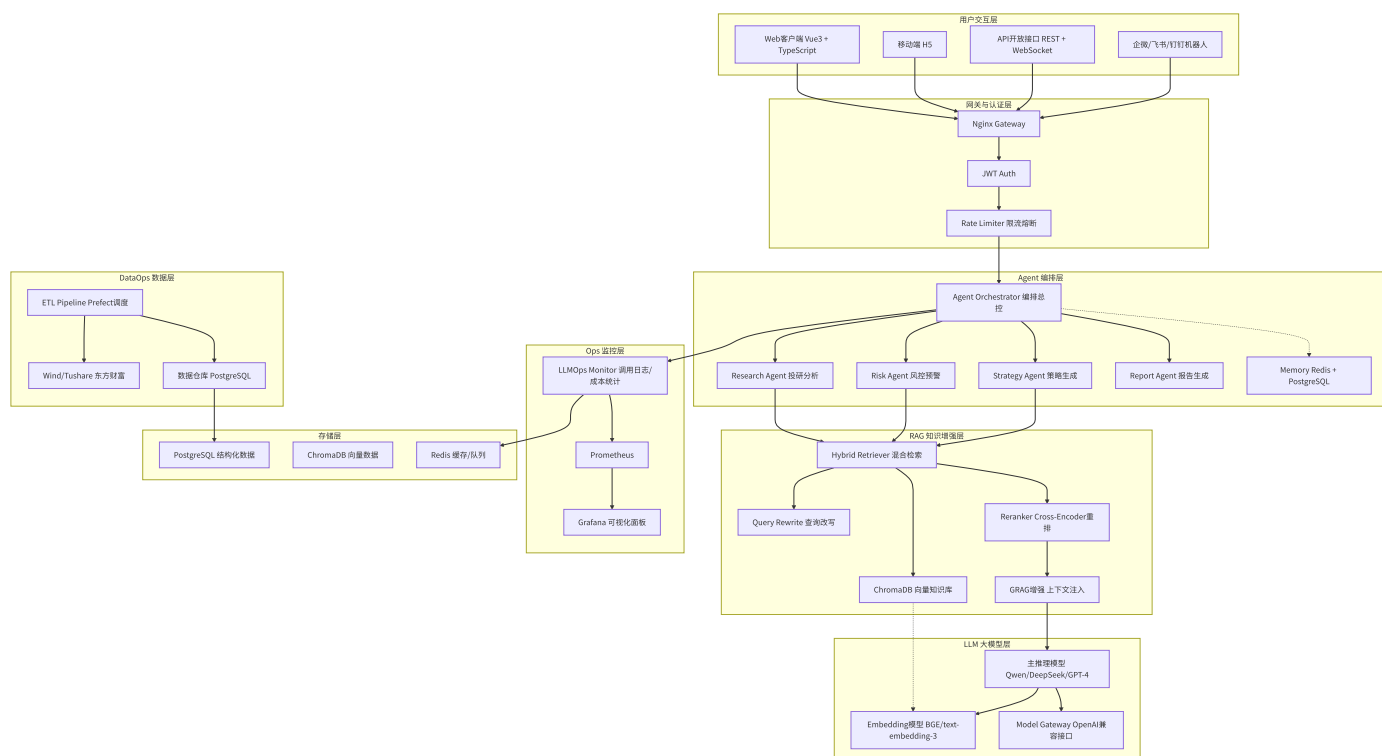
1.1 项目背景

随着金融市场的日益复杂化和数据量的爆发式增长，传统的人工分析模式已无法满足投资者对实时性、精准性和深度洞察的需求。本项目构建了一个**基于大语言模型(LLM)的智能金融分析AI Agent**，融合RAG检索增强、Agent多智能体协作、LLMOps模型运维、DataOps数据编排等前沿技术，为用户提供股票分析、财报解读、市场研判、风险预警等一站式智能投研服务。

1.2 核心功能

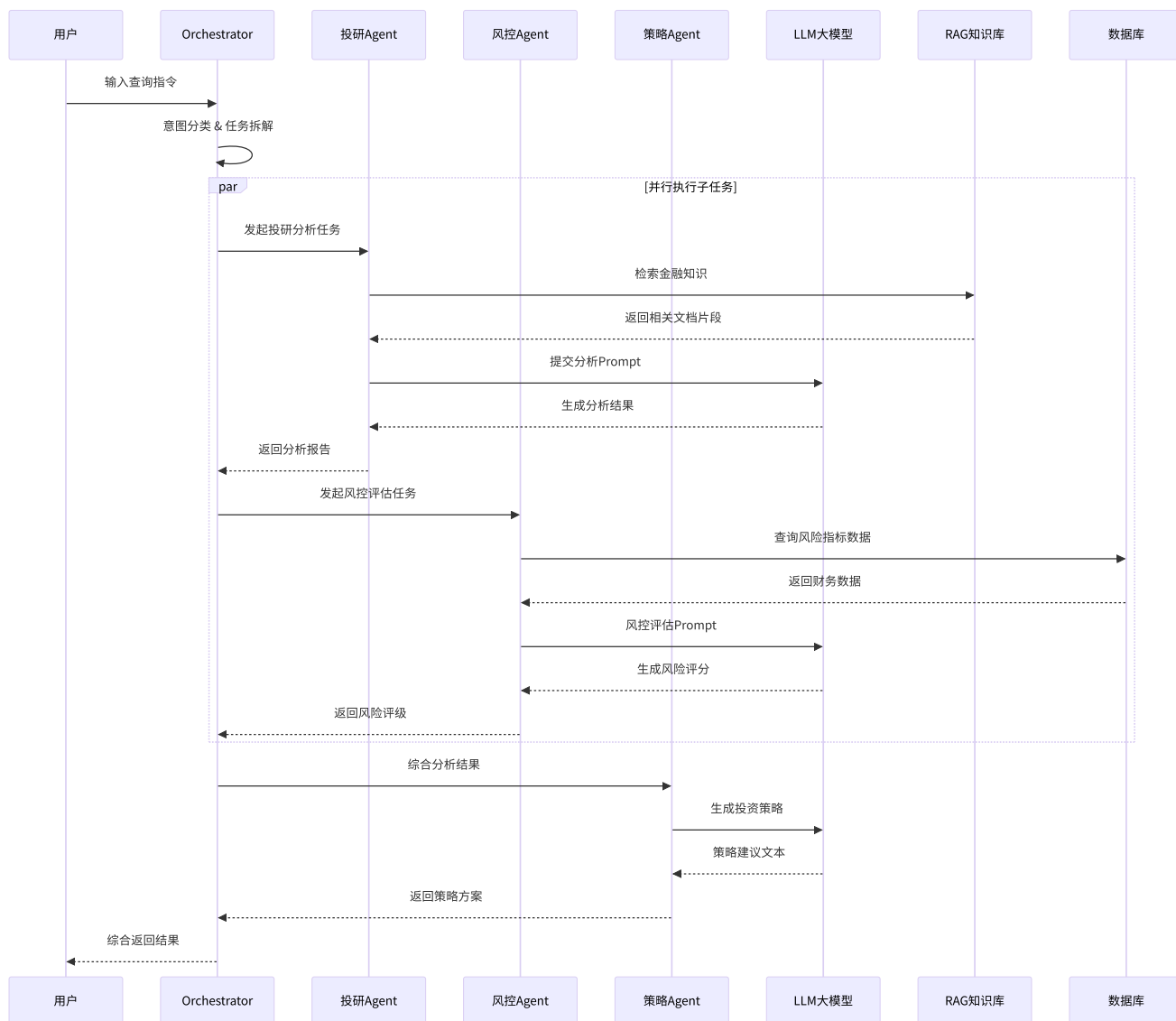
模块	功能描述
智能投研助手	自然语言查询股票基本面、财务指标、估值分析
RAG知识库问答	基于金融知识库(法规、年报、研报)的精准问答
多Agent协作分析	分析师Agent + 风控Agent + 策略Agent 协同工作
市场实时监控	实时行情、异动预警、舆情监控
数据可视化看板	交互式图表、K线图、财务报表可视化
LLMOps监控面板	模型调用统计、Token消耗、响应延迟监控

2. 系统架构图

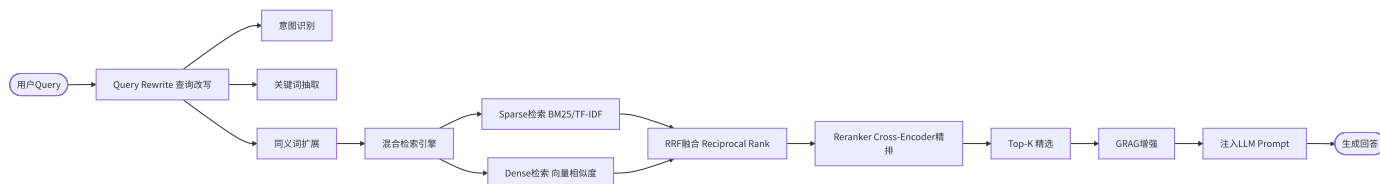


3. 核心流程图

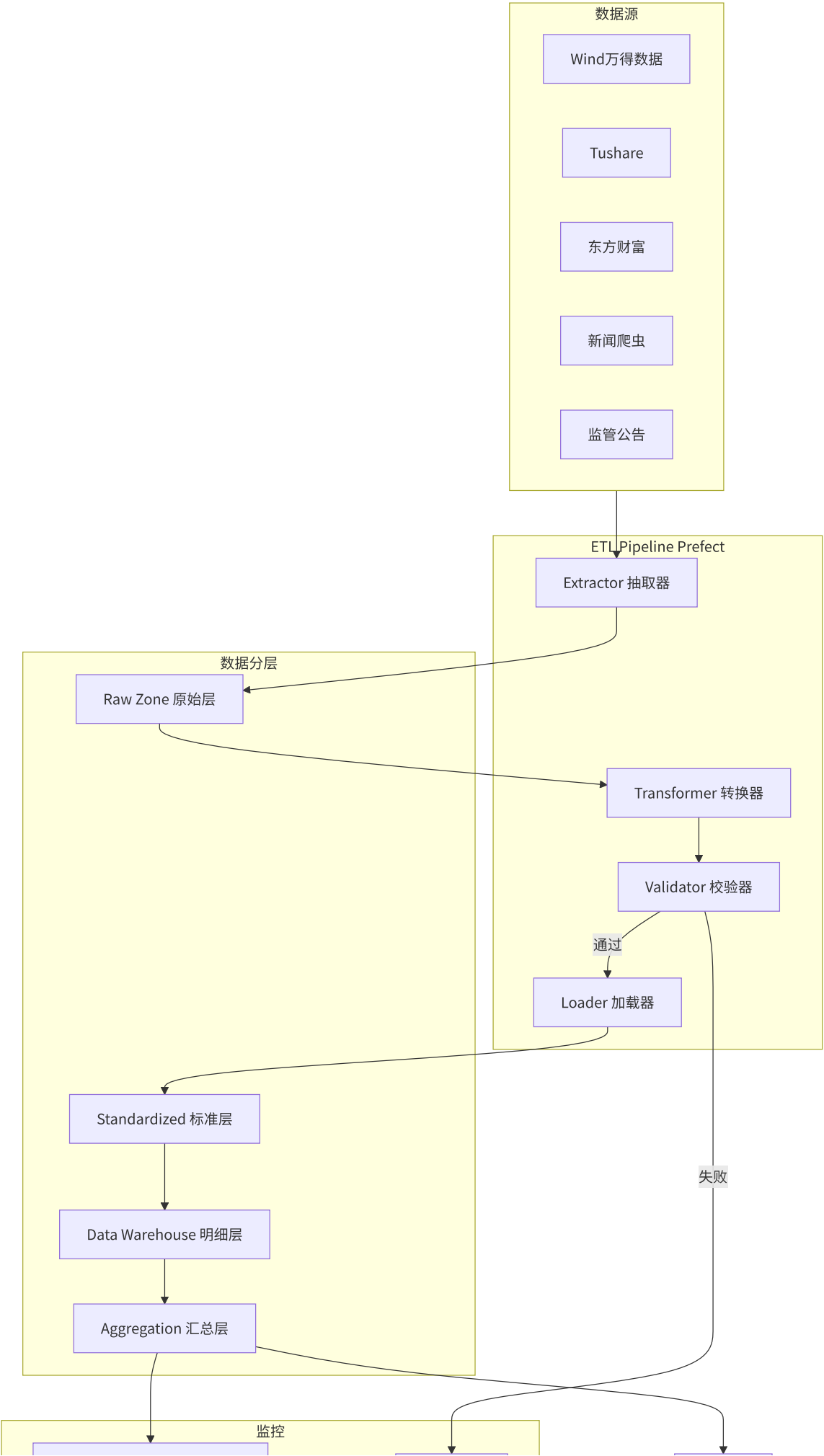
3.1 Agent 协作时序图

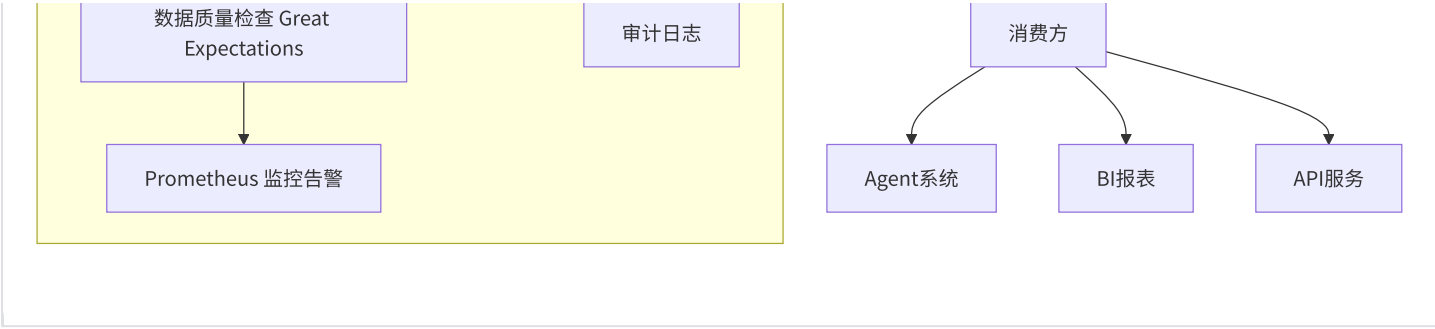


3.2 RAG 检索流程图



3.3 DataOps 数据管道流程





4. 技术选型

分类	技术选型	说明
LLM大模型	Qwen / DeepSeek / GPT-4	主推理模型，OpenAI兼容接口
Embedding	BGE-large-zh / text-embedding-3-large	文本向量化，1536维
向量数据库	ChromaDB (本地) / Milvus (生产)	RAG向量检索引擎
Agent框架	LangChain + 自研编排器	多Agent协同、工具调用
后端框架	FastAPI + Python 3.11	高性能异步API服务
前端框架	Vue3 + TypeScript + Vite	响应式深色主题UI
数据库	PostgreSQL + TimescaleDB	结构化存储 + 时序优化
缓存/队列	Redis	会话缓存、消息队列
数据管道	Prefect	ETL编排调度
监控	Prometheus + Grafana	指标采集可视化
容器化	Docker + Docker Compose	一键部署

5. 项目结构

代码块

```
1 Financial-AI-Agent/  
2 |— docs/  
3 |   └─ 项目技术文档.md  
4 |— INTEGRATED.md  
5 |— backend/                                ← FastAPI 后端
```

```

6 | | | └─ app/
7 | | |   | └─ main.py           FastAPI入口
8 | | |   | └─ config.py        全局配置
9 | | |   | └─ core/
10 | | |   |   | └─ llm.py        LLM统一接口
11 | | |   |   | └─ redis_client.py  Redis连接
12 | | |   |   | └─ vectorstore.py  向量库接口
13 | | |   | └─ models/
14 | | |   |   | └─ schemas.py      Pydantic模型
15 | | |   |   | └─ database.py     SQLAlchemy ORM
16 | | |   | └─ agents/
17 | | |   |   | └─ base.py         Agent基类
18 | | |   |   | └─ orchestrator.py 编排器
19 | | |   |   | └─ researcher.py   投研Agent
20 | | |   |   | └─ risk_agent.py   风控Agent
21 | | |   |   | └─ strategist.py   策略Agent
22 | | |   |   | └─ report_agent.py 报告Agent
23 | | |   | └─ rag/
24 | | |   |   | └─ knowledge_base.py 知识库
25 | | |   |   | └─ retriever.py      混合检索
26 | | |   |   | └─ query_rewrite.py  查询改写
27 | | |   | └─ llmops/
28 | | |   |   | └─ monitor.py        监控核心
29 | | |   | └─ dataops/
30 | | |   |   | └─ pipeline.py       数据管道
31 | | |   | └─ api/
32 | | |   |   | └─ chat.py           对话API
33 | | |   |   | └─ agent.py         AgentAPI
34 | | |   |   | └─ rag.py           RAG API
35 | | |   |   | └─ llmops.py        LLMOps API
36 | | |   |   | └─ data.py          数据API
37 | | | └─ requirements.txt
38 | | └─ Dockerfile
39 | └─ frontend/                ← Vue3 前端
40 |   | └─ package.json
41 |   | └─ vite.config.ts
42 |   | └─ tsconfig.json
43 |   | └─ index.html
44 |   | └─ src/
45 |   |   | └─ main.ts
46 |   |   | └─ App.vue
47 |   |   | └─ router.ts
48 |   |   | └─ style.css
49 |   |   | └─ api/
50 |   |   |   | └─ index.ts
51 |   |   |   | └─ chat.ts
52 |   |   |   | └─ agent.ts

```

```
53 |         | └─ llmops.ts
54 |         └─ stores/
55 |         |   └─ chat.ts
56 |         |   └─ llmops.ts
57 |         └─ views/
58 |         |   └─ ChatView.vue
59 |         |   └─ DashboardView.vue
60 |         |   └─ ReportView.vue
61 |         |   └─ KnowledgeView.vue
62 |         |   └─ LLMOpsView.vue
63 | └─ docker-compose.yml
64 | └─ .env.example
```

6. 后端完整代码

6.1 全局配置管理

代码块

```
1  """
2  backend/app/config.py
3  全局配置管理 - 使用 Pydantic Settings
4  """
5  from pydantic_settings import BaseSettings
6  from typing import Optional
7  from functools import lru_cache
8
9
10 class Settings(BaseSettings):
11     """应用全局配置"""
12
13     # === 应用基础配置 ===
14     APP_NAME: str = "Financial AI Agent"
15     APP_VERSION: str = "1.0.0"
16     DEBUG: bool = False
17     API_PREFIX: str = "/api/v1"
18
19     # === FastAPI 服务配置 ===
20     HOST: str = "0.0.0.0"
21     PORT: int = 8000
22     WORKERS: int = 4
23
24     # === LLM 大模型配置 ===
25     LLM_PROVIDER: str = "openai"
```

```
26 LLM_MODEL: str = "qwen-plus"
27 LLM_API_BASE: str = "https://dashscope.aliyuncs.com/compatible-mode/v1"
28 LLM_API_KEY: str = ""
29 LLM_TEMPERATURE: float = 0.7
30 LLM_MAX_TOKENS: int = 4096
31 LLM_TIMEOUT: int = 120
32
33 # === 嵌入模型配置 ===
34 EMBEDDING_PROVIDER: str = "openai"
35 EMBEDDING_MODEL: str = "text-embedding-3-large"
36 EMBEDDING_API_BASE: str = "https://dashscope.aliyuncs.com/compatible-
mode/v1"
37 EMBEDDING_API_KEY: str = ""
38 EMBEDDING_BATCH_SIZE: int = 100
39 EMBEDDING_DIM: int = 1536
40
41 # === 数据库配置 ===
42 DATABASE_URL: str =
"postgresql+asyncpg://user:pass@localhost:5432/financial_agent"
43 DATABASE_POOL_SIZE: int = 20
44 DATABASE_MAX_OVERFLOW: int = 10
45
46 # === 向量数据库配置 ===
47 VECTORSTORE_TYPE: str = "chroma"
48 CHROMA_PERSIST_DIR: str = "./data/chromadb"
49 CHROMA_COLLECTION_NAME: str = "financial_knowledge"
50
51 # === Redis 配置 ===
52 REDIS_HOST: str = "localhost"
53 REDIS_PORT: int = 6379
54 REDIS_DB: int = 0
55 REDIS_PASSWORD: Optional[str] = None
56 REDIS_MAX_CONNECTIONS: int = 50
57
58 # === RAG 配置 ===
59 RAG_TOP_K: int = 10
60 RAG_SCORE_THRESHOLD: float = 0.5
61 RAG_RERANK_TOP_K: int = 5
62 RAG_MAX_CONTEXT_LENGTH: int = 8000
63
64 # === Agent 配置 ===
65 AGENT_MAX_ITERATIONS: int = 10
66 AGENT_EARLY_STOP_THRESHOLD: float = 0.8
67 AGENT_TOOL_CALL_TIMEOUT: int = 30
68 AGENT_MEMORY_TYPE: str = "buffer"
69
70 # === DataOps 配置 ===
```

```

71 DATAOPS_PREFECT_API: str = "http://localhost:4200/api"
72 DATAOPS_WIND_TOKEN: str = ""
73 DATAOPS_TUSHARE_TOKEN: str = ""
74 DATAOPS_ETL_SCHEDULE_CRON: str = "0 2 * * *"
75
76 # === LLMOps 配置 ===
77 LLMOPS_PROMETHEUS_PORT: int = 9090
78 LLMOPS_ENABLE_METRICS: bool = True
79 LLMOPS_LOG_FILE: str = "./logs/llm_requests.log"
80
81 # === 安全配置 ===
82 SECRET_KEY: str = "change-this-in-production"
83 ALGORITHM: str = "HS256"
84 ACCESS_TOKEN_EXPIRE_MINUTES: int = 60
85 CORS_ORIGINS: list[str] = ["http://localhost:3000",
86 "http://localhost:5173"]
87
88 class Config:
89     env_file = ".env"
90     env_file_encoding = "utf-8"
91     extra = "allow"
92
93 @lru_cache()
94 def get_settings() -> Settings:
95     """获取单例配置实例"""
96     return Settings()

```

6.2 LLM 大模型统一调用接口

代码块

```

1  """
2  backend/app/core/llm.py
3  LLM 大模型统一调用接口 - 支持多模型供应商
4  """
5  import time
6  import logging
7  from typing import Optional, AsyncIterator, Any, Dict, List
8  from dataclasses import dataclass
9
10 from langchain_openai import ChatOpenAI, OpenAIEmbeddings
11 from langchain.schema import BaseMessage, HumanMessage, SystemMessage
12
13 from app.config import get_settings
14

```

```
15 logger = logging.getLogger(__name__)
16
17
18 @dataclass
19 class LLMCallLog:
20     """LLM 调用日志"""
21     model: str
22     prompt_tokens: int
23     completion_tokens: int
24     total_tokens: int
25     latency_ms: float
26     cost_usd: float
27     timestamp: str
28     success: bool
29     error: Optional[str] = None
30
31
32 class TokenCostCalculator:
33     """Token 成本计算器"""
34
35     GPT40_INPUT = 0.0025 / 1000
36     GPT40_OUTPUT = 0.01 / 1000
37     QWEN_INPUT = 0.001 / 1000
38     QWEN_OUTPUT = 0.002 / 1000
39     DEEPSEEK_INPUT = 0.001 / 1000
40     DEEPSEEK_OUTPUT = 0.002 / 1000
41
42     @classmethod
43     def calculate(cls, model: str, input_tokens: int, output_tokens: int) ->
float:
44         model_lower = model.lower()
45         if "gpt-4o" in model_lower or "gpt-4" in model_lower:
46             return input_tokens * cls.GPT40_INPUT + output_tokens *
cls.GPT40_OUTPUT
47         elif "qwen" in model_lower:
48             return input_tokens * cls.QWEN_INPUT + output_tokens *
cls.QWEN_OUTPUT
49         elif "deepseek" in model_lower:
50             return input_tokens * cls.DEEPSEEK_INPUT + output_tokens *
cls.DEEPSEEK_OUTPUT
51         return 0.0
52
53
54 class LLMService:
55     """LLM 大模型统一服务接口"""
56
57     def __init__(self):
```

```
58     self.settings = get_settings()
59     self._llm: Optional[ChatOpenAI] = None
60     self._embeddings: Optional[OpenAIEmbeddings] = None
61
62     @property
63     def llm(self) -> ChatOpenAI:
64         """延迟初始化 LLM 客户端"""
65         if self._llm is None:
66             self._llm = ChatOpenAI(
67                 model=self.settings.LLM_MODEL,
68                 api_key=self.settings.LLM_API_KEY,
69                 base_url=self.settings.LLM_API_BASE,
70                 temperature=self.settings.LLM_TEMPERATURE,
71                 max_tokens=self.settings.LLM_MAX_TOKENS,
72                 timeout=self.settings.LLM_TIMEOUT,
73                 streaming=True,
74             )
75         return self._llm
76
77     @property
78     def embeddings(self) -> OpenAIEmbeddings:
79         """延迟初始化 Embedding 客户端"""
80         if self._embeddings is None:
81             self._embeddings = OpenAIEmbeddings(
82                 model=self.settings.EMBEDDING_MODEL,
83                 api_key=self.settings.EMBEDDING_API_KEY,
84                 base_url=self.settings.EMBEDDING_API_BASE,
85                 batch_size=self.settings.EMBEDDING_BATCH_SIZE,
86             )
87         return self._embeddings
88
89     async def chat(
90         self,
91         messages: List[BaseMessage],
92         tools: Optional[List[Dict]] = None,
93         tool_choice: Optional[str] = None,
94         stream: bool = True,
95     ) -> LLMCallLog:
96         """通用对话接口"""
97         start_time = time.time()
98
99         try:
100             if tools:
101                 response = await self.llm.bind_tools(tools).ainvoke(messages)
102             else:
103                 response = await self.llm.ainvoke(messages)
104
```

```

105         latency = (time.time() - start_time) * 1000
106
107         usage = getattr(response, "usage", None)
108         if usage:
109             prompt_tokens = getattr(usage, "prompt_tokens", 0)
110             completion_tokens = getattr(usage, "completion_tokens", 0)
111             total_tokens = getattr(usage, "total_tokens", 0)
112         else:
113             prompt_tokens = sum(len(str(m.content)) for m in messages) // 4
114             completion_tokens = len(str(response.content)) // 4
115             total_tokens = prompt_tokens + completion_tokens
116
117         cost = TokenCostCalculator.calculate(
118             self.settings.LLM_MODEL, prompt_tokens, completion_tokens
119         )
120
121         log = LLMCallLog(
122             model=self.settings.LLM_MODEL,
123             prompt_tokens=prompt_tokens,
124             completion_tokens=completion_tokens,
125             total_tokens=total_tokens,
126             latency_ms=latency,
127             cost_usd=cost,
128             timestamp=time.strftime("%Y-%m-%d %H:%M:%S"),
129             success=True,
130         )
131
132         logger.info(
133             f"[LLM] model={self.settings.LLM_MODEL} tokens={total_tokens} "
134             f"latency={latency:.0f}ms cost=${cost:.6f}"
135         )
136         return log
137
138     except Exception as e:
139         latency = (time.time() - start_time) * 1000
140         logger.error(f"[LLM] 调用失败: {str(e)}")
141         return LLMCallLog(
142             model=self.settings.LLM_MODEL,
143             prompt_tokens=0, completion_tokens=0, total_tokens=0,
144             latency_ms=latency, cost_usd=0.0,
145             timestamp=time.strftime("%Y-%m-%d %H:%M:%S"),
146             success=False, error=str(e),
147         )
148
149     async def chat_stream(
150         self, messages: List[BaseMessage]
151     ) -> AsyncIterator[tuple[str, Optional[LLMCallLog]]]:

```

```

152         """流式对话"""
153         start_time = time.time()
154         full_response = ""
155
156         try:
157             async for chunk in self.llm.astream(messages):
158                 token = chunk.content if hasattr(chunk, "content") else
str(chunk)
159                 full_response += token
160                 yield token, None
161
162                 latency = (time.time() - start_time) * 1000
163                 prompt_tokens = sum(len(str(m.content)) for m in messages) // 4
164                 completion_tokens = len(full_response) // 4
165                 total_tokens = prompt_tokens + completion_tokens
166                 cost = TokenCostCalculator.calculate(
167                     self.settings.LLM_MODEL, prompt_tokens, completion_tokens
168                 )
169
170                 log = LLMCallLog(
171                     model=self.settings.LLM_MODEL,
172                     prompt_tokens=prompt_tokens,
173                     completion_tokens=completion_tokens,
174                     total_tokens=total_tokens,
175                     latency_ms=latency,
176                     cost_usd=cost,
177                     timestamp=time.strftime("%Y-%m-%d %H:%M:%S"),
178                     success=True,
179                 )
180                 yield "", log
181
182         except Exception as e:
183             latency = (time.time() - start_time) * 1000
184             log = LLMCallLog(
185                 model=self.settings.LLM_MODEL,
186                 prompt_tokens=0, completion_tokens=0, total_tokens=0,
187                 latency_ms=latency, cost_usd=0.0,
188                 timestamp=time.strftime("%Y-%m-%d %H:%M:%S"),
189                 success=False, error=str(e),
190             )
191             yield "", log
192
193     async def embed(self, texts: List[str]) -> List[List[float]]:
194         """文本向量化"""
195         try:
196             embeddings = await self.embeddings.aembed_documents(texts)

```

```

197         logger.info(f"[Embedding] texts={len(texts)} dim=
{len(embeddings[0]) if embeddings else 0}")
198         return embeddings
199     except Exception as e:
200         logger.error(f"[Embedding] 失败: {str(e)}")
201         raise
202
203
204 llm_service = LLMService()
205
206
207 def get_llm_service() -> LLMService:
208     return llm_service

```

6.3 Pydantic 数据模型定义

代码块

```

1  """
2  backend/app/models/schemas.py
3  Pydantic 请求/响应数据模型
4  """
5  from pydantic import BaseModel, Field
6  from typing import Optional, List, Literal, Any, Dict
7  from datetime import datetime
8  from enum import Enum
9
10
11 class AgentType(str, Enum):
12     RESEARCHER = "researcher"
13     RISK = "risk"
14     STRATEGIST = "strategist"
15     REPORT = "report"
16
17
18 class TaskStatus(str, Enum):
19     PENDING = "pending"
20     RUNNING = "running"
21     COMPLETED = "completed"
22     FAILED = "failed"
23
24
25 class ChatMessage(BaseModel):
26     role: Literal["user", "assistant", "system", "tool"]
27     content: str
28     agent_type: Optional[AgentType] = None

```

```
29     timestamp: datetime = Field(default_factory=datetime.now)
30     metadata: Optional[Dict[str, Any]] = None
31
32
33     class ChatRequest(BaseModel):
34         session_id: str
35         message: str = Field(..., min_length=1, max_length=4000)
36         agent_type: Optional[AgentType] = None
37         stream: bool = True
38         enable_rag: bool = True
39
40
41     class ChatResponse(BaseModel):
42         session_id: str
43         message: str
44         agent_type: AgentType
45         sources: Optional[List[Dict[str, Any]]] = None
46         token_usage: Optional[Dict[str, int]] = None
47         latency_ms: Optional[float] = None
48         session_created: bool = False
49
50
51     class AgentInvokeRequest(BaseModel):
52         session_id: str
53         query: str
54         agent_types: Optional[List[AgentType]] = None
55         parallel: bool = True
56
57
58     class RAGQueryRequest(BaseModel):
59         query: str
60         top_k: int = Field(default=10, ge=1, le=100)
61         score_threshold: float = Field(default=0.5, ge=0, le=1)
62         filters: Optional[Dict[str, Any]] = None
63         enable_rerank: bool = True
64
65
66     class RAGQueryResponse(BaseModel):
67         query: str
68         chunks: List[Dict[str, Any]]
69         total_retrieved: int
70         rerank_applied: bool
71
72
73     class KnowledgeBaseUpload(BaseModel):
74         title: str
75         content: str
```

```

76     category: str
77     metadata: Optional[Dict[str, Any]] = None
78
79
80     class LLMCallRecord(BaseModel):
81         call_id: str
82         model: str
83         prompt_tokens: int
84         completion_tokens: int
85         total_tokens: int
86         latency_ms: float
87         cost_usd: float
88         success: bool
89         timestamp: str
90
91
92     class LLMOpsMetrics(BaseModel):
93         all_time: Dict[str, Any]
94         today: Dict[str, Any]
95
96
97     class DataPipelineRun(BaseModel):
98         run_id: str
99         pipeline_name: str
100         status: TaskStatus
101         started_at: str
102         completed_at: Optional[str] = None
103         records_processed: int = 0
104         records_failed: int = 0
105         error_message: Optional[str] = None

```

6.4 SQLAlchemy ORM 数据库模型

代码块

```

1  """
2  backend/app/models/database.py
3  SQLAlchemy ORM 模型 - PostgreSQL 数据库表定义
4  """
5  from sqlalchemy import (
6      Column, String, Integer, Float, Boolean, DateTime, Text, JSON, Index,
7      ForeignKey, UniqueConstraint
8  )
9  from sqlalchemy.orm import DeclarativeBase, relationship
10 from sqlalchemy.sql import func
11

```

```
12
13 class Base(DeclarativeBase):
14     pass
15
16
17 class User(Base):
18     __tablename__ = "users"
19
20     id = Column(Integer, primary_key=True, index=True)
21     username = Column(String(50), unique=True, index=True, nullable=False)
22     email = Column(String(100), unique=True, index=True)
23     hashed_password = Column(String(255), nullable=False)
24     full_name = Column(String(100))
25     is_active = Column(Boolean, default=True)
26     is_superuser = Column(Boolean, default=False)
27     created_at = Column(DateTime(timezone=True), server_default=func.now())
28     updated_at = Column(DateTime(timezone=True), onupdate=func.now())
29
30     sessions = relationship("ChatSession", back_populates="user",
31                             cascade="all, delete-orphan")
32     llm_calls = relationship("LLMCallLog", back_populates="user",
33                             cascade="all, delete-orphan")
34
35
36
37 class ChatSession(Base):
38     __tablename__ = "chat_sessions"
39
40     id = Column(Integer, primary_key=True, index=True)
41     session_id = Column(String(100), unique=True, index=True, nullable=False)
42     user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
43     title = Column(String(200), default="新对话")
44     agent_types = Column(JSON, default=list)
45     created_at = Column(DateTime(timezone=True), server_default=func.now())
46     updated_at = Column(DateTime(timezone=True), onupdate=func.now())
47     last_message = Column(Text)
48     message_count = Column(Integer, default=0)
49
50     user = relationship("User", back_populates="sessions")
51     messages = relationship("ChatMessageDB", back_populates="session",
52                             cascade="all, delete-orphan")
53
54     __table_args__ = (Index("idx_session_created", "session_id",
55                             "created_at"),)
56
57
58
59 class ChatMessageDB(Base):
60     __tablename__ = "chat_messages"
```

```

55
56     id = Column(Integer, primary_key=True, index=True)
57     session_id = Column(Integer, ForeignKey("chat_sessions.id"),
nullable=False)
58     role = Column(String(20), nullable=False)
59     content = Column(Text, nullable=False)
60     agent_type = Column(String(50))
61     token_count = Column(Integer, default=0)
62     metadata = Column(JSON, default=dict)
63     created_at = Column(DateTime(timezone=True), server_default=func.now())
64
65     session = relationship("ChatSession", back_populates="messages")
66
67
68 class LLMCallLog(Base):
69     """LLM 调用日志"""
70     __tablename__ = "llm_call_logs"
71
72     id = Column(Integer, primary_key=True, index=True)
73     call_id = Column(String(100), unique=True, index=True, nullable=False)
74     session_id = Column(String(100), index=True)
75     user_id = Column(Integer, ForeignKey("users.id"), nullable=True)
76     model = Column(String(50), nullable=False)
77     prompt_tokens = Column(Integer, default=0)
78     completion_tokens = Column(Integer, default=0)
79     total_tokens = Column(Integer, default=0)
80     latency_ms = Column(Float, default=0)
81     cost_usd = Column(Float, default=0)
82     success = Column(Boolean, default=True)
83     error_message = Column(Text)
84     prompt_preview = Column(Text)
85     response_preview = Column(Text)
86     metadata = Column(JSON, default=dict)
87     created_at = Column(DateTime(timezone=True), server_default=func.now())
88
89     user = relationship("User", back_populates="llm_calls")
90
91     __table_args__ = (
92         Index("idx_llm_model_created", "model", "created_at"),
93         Index("idx_llm_daily", "created_at"),
94     )
95
96
97 class PromptVersionDB(Base):
98     """Prompt 版本管理"""
99     __tablename__ = "prompt_versions"
100

```

```
101     id = Column(Integer, primary_key=True, index=True)
102     version_id = Column(String(100), unique=True, index=True, nullable=False)
103     name = Column(String(100), nullable=False)
104     agent_type = Column(String(50))
105     content = Column(Text, nullable=False)
106     variables = Column(JSON, default=list)
107     is_active = Column(Boolean, default=False)
108     created_by = Column(Integer, ForeignKey("users.id"))
109     created_at = Column(DateTime(timezone=True), server_default=func.now())
110
111
112 class KnowledgeDocument(Base):
113     """知识库文档"""
114     __tablename__ = "knowledge_documents"
115
116     id = Column(Integer, primary_key=True, index=True)
117     doc_id = Column(String(100), unique=True, index=True, nullable=False)
118     title = Column(String(500), nullable=False)
119     category = Column(String(50), nullable=False)
120     source = Column(String(200))
121     content_hash = Column(String(64))
122     metadata = Column(JSON, default=dict)
123     chunk_count = Column(Integer, default=0)
124     indexed = Column(Boolean, default=False)
125     created_at = Column(DateTime(timezone=True), server_default=func.now())
126     updated_at = Column(DateTime(timezone=True), onupdate=func.now())
127
128     chunks = relationship("DocumentChunkDB", back_populates="document",
129                           cascade="all, delete-orphan")
130
131 class DocumentChunkDB(Base):
132     """文档片段"""
133     __tablename__ = "document_chunks"
134
135     id = Column(Integer, primary_key=True, index=True)
136     chunk_id = Column(String(100), unique=True, index=True, nullable=False)
137     document_id = Column(Integer, ForeignKey("knowledge_documents.id"),
138                          nullable=False)
139     content = Column(Text, nullable=False)
140     chunk_index = Column(Integer, default=0)
141     metadata = Column(JSON, default=dict)
142     embedding_id = Column(String(200))
143     created_at = Column(DateTime(timezone=True), server_default=func.now())
144
145     document = relationship("KnowledgeDocument", back_populates="chunks")
```

```

146     __table_args__ = (Index("idx_chunk_doc_index", "document_id",
147                             "chunk_index"),)
148
149     class DataPipeline(Base):
150         """数据管道定义"""
151         __tablename__ = "data_pipelines"
152
153         id = Column(Integer, primary_key=True, index=True)
154         pipeline_name = Column(String(100), unique=True, nullable=False)
155         description = Column(Text)
156         source_type = Column(String(50))
157         schedule_cron = Column(String(100))
158         is_active = Column(Boolean, default=True)
159         config = Column(JSON, default=dict)
160         last_run_at = Column(DateTime(timezone=True))

```

6.5 Agent 基类框架

代码块

```

1     """
2     backend/app/agents/base.py
3     Agent 基类 - 基于 LangChain 实现通用 Agent 框架
4     """
5     import uuid
6     import time
7     import logging
8     from typing import Dict, Any, List, Optional, AsyncIterator
9     from dataclasses import dataclass, field
10    from abc import ABC, abstractmethod
11
12    from langchain.schema import HumanMessage, SystemMessage, AIMessage,
13    BaseMessage
14
15    from langchain.tools import BaseTool
16
17    from app.models.schemas import AgentType
18    from app.core.llm import LLMService, get_llm_service
19
20
21    logger = logging.getLogger(__name__)
22
23    @dataclass
24    class AgentConfig:
25        name: str
26        agent_type: AgentType

```

```

25     description: str
26     system_prompt: str
27     tools: List[BaseTool] = field(default_factory=list)
28     max_iterations: int = 10
29     early_stop_threshold: float = 0.8
30     temperature: float = 0.7
31
32
33 @dataclass
34 class AgentExecutionResult:
35     success: bool
36     output: str
37     tool_calls: List[Dict[str, Any]] = field(default_factory=list)
38     iterations: int = 0
39     total_tokens: int = 0
40     execution_time_ms: float = 0.0
41     error: Optional[str] = None
42
43
44 class BaseAgent(ABC):
45     """
46     Agent 基类 - 提供通用生命周期管理
47     """
48
49     def __init__(self, config: AgentConfig, llm_service: Optional[LLMService]
= None):
50         self.config = config
51         self.llm = llm_service or get_llm_service()
52         self._execution_history: List[Dict] = []
53         self._memory: List[BaseMessage] = []
54         self.system_prompt = SystemMessage(content=config.system_prompt)
55         logger.info(f"[Agent] {config.name} initialized, tools=
{len(config.tools)}")
56
57     @abstractmethod
58     def _build_system_prompt(self) -> str:
59         pass
60
61     @abstractmethod
62     def _parse_output(self, raw_output: str) -> Dict[str, Any]:
63         pass
64
65     async def execute(
66         self, task: str,
67         context: Optional[Dict[str, Any]] = None,
68         session_id: Optional[str] = None,
69     ) -> AgentExecutionResult:

```

```

70     task_id = str(uuid.uuid4())
71     start_time = time.time()
72     total_tokens = 0
73
74     logger.info(f"[Agent:{self.config.name}] executing task_id={task_id}")
75
76     try:
77         prompt_content = self._build_system_prompt()
78         if context:
79             context_str = self._format_context(context)
80             prompt_content += f"\n\n## Context\n{context_str}"
81             prompt_content += f"\n\n## Task\n{task}"
82
83         messages = [
84             SystemMessage(content=prompt_content),
85             HumanMessage(content=task),
86         ]
87
88         tools_json = self._tools_to_json(self.config.tools)
89         if tools_json:
90             call_log = await self.llm.chat(messages=messages,
91             tools=tools_json, stream=False)
92             if not call_log.success:
93                 raise Exception(f"LLM failed: {call_log.error}")
94             total_tokens = call_log.total_tokens
95             agent_output = f"[tools executed]\n{call_log.error or 'done'}"
96         else:
97             call_log = await self.llm.chat(messages=messages, stream=False)
98             if not call_log.success:
99                 raise Exception(f"LLM failed: {call_log.error}")
100             total_tokens = call_log.total_tokens
101             agent_output = str(call_log.error or "done")
102
103         parsed = self._parse_output(agent_output)
104         execution_time = (time.time() - start_time) * 1000
105
106         self._execution_history.append({
107             "task_id": task_id, "task": task, "output": agent_output,
108             "parsed": parsed, "tokens": total_tokens,
109             "execution_time_ms": execution_time,
110             "timestamp": time.strftime("%Y-%m-%d %H:%M:%S"),
111         })
112
113         return AgentExecutionResult(
114             success=True, output=agent_output, iterations=1,
115             total_tokens=total_tokens, execution_time_ms=execution_time,

```

```

116
117         except Exception as e:
118             execution_time = (time.time() - start_time) * 1000
119             logger.error(f"[Agent:{self.config.name}] failed: {str(e)}",
exc_info=True)
120             return AgentExecutionResult(
121                 success=False, output="", error=str(e),
122                 iterations=0, total_tokens=total_tokens,
123                 execution_time_ms=execution_time,
124             )
125
126     async def execute_stream(
127         self, task: str, context: Optional[Dict[str, Any]] = None
128     ) -> AsyncIterator[str]:
129         prompt_content = self._build_system_prompt()
130         if context:
131             context_str = self._format_context(context)
132             prompt_content += f"\n\n## Context\n{context_str}"
133             prompt_content += f"\n\n## Task\n{task}"
134             messages = [SystemMessage(content=prompt_content),
HumanMessage(content=task)]
135             full = ""
136             async for token, _ in self.llm.chat_stream(messages):
137                 full += token
138                 yield token
139             logger.info(f"[Agent:{self.config.name}] stream done, len={len(full)}")
140
141     def _format_context(self, context: Dict[str, Any]) -> str:
142         lines = []
143         for key, value in context.items():
144             if isinstance(value, list):
145                 lines.append(f"### {key}")
146                 for item in value:
147                     if isinstance(item, dict):
148                         lines.append(f"- {item.get('content', str(item))
[:200]}")
149                     else:
150                         lines.append(f"- {str(item)[:200]}")
151             elif isinstance(value, dict):
152                 lines.append(f"### {key}")
153                 for k, v in value.items():
154                     lines.append(f"- {k}: {str(v)[:200]}")
155             else:
156                 lines.append(f"**{key}**: {str(value)[:500]}")
157         return "\n".join(lines)
158
159     def _tools_to_json(self, tools: List[BaseTool]) -> List[Dict]:

```

```

160         result = []
161         for tool in tools:
162             result.append({
163                 "type": "function",
164                 "function": {
165                     "name": tool.name,
166                     "description": tool.description,
167                     "parameters": {"type": "object", "properties": {}},
168                 },
169             })
170         return result
171
172     def get_execution_history(self) -> List[Dict]:
173         return self._execution_history
174
175     def clear_history(self):
176         self._execution_history.clear()

```

6.6 多Agent智能编排器

代码块

```

1  """
2  backend/app/agents/orchestrator.py
3  Agent 编排器 - 多Agent协同调度核心
4  """
5  import uuid
6  import time
7  import logging
8  import re
9  from typing import Dict, Any, List, Optional
10 from dataclasses import dataclass
11 import asyncio
12
13 from app.agents.base import AgentExecutionResult
14 from app.agents.researcher import ResearcherAgent
15 from app.agents.risk_agent import RiskAgent
16 from app.agents.strategist import StrategistAgent
17 from app.models.schemas import AgentType
18
19 logger = logging.getLogger(__name__)
20
21
22 @dataclass
23 class OrchestratorConfig:

```

```

24     max_parallel_agents: int = 3
25     timeout_per_agent: int = 60
26     enable_caching: bool = True
27
28
29     class AgentOrchestrator:
30         """
31         多Agent编排器
32         核心能力: 意图分类 | 任务拆解 | 并行/串行执行 | 结果聚合 | 记忆管理
33         """
34
35         INTENT_AGENT_MAP = {
36             "stock_analysis": [AgentType.RESEARCHER],
37             "stock_risk": [AgentType.RISK],
38             "full_analysis": [AgentType.RESEARCHER, AgentType.RISK,
AgentType.STRATEGIST],
39             "investment_advice": [AgentType.RESEARCHER, AgentType.STRATEGIST],
40             "risk_warning": [AgentType.RISK],
41             "report": [AgentType.RESEARCHER],
42             "general": [AgentType.RESEARCHER],
43         }
44
45         def __init__(self, config: Optional[OrchestratorConfig] = None):
46             self.config = config or OrchestratorConfig()
47             self._agents: Dict[AgentType, Any] = {}
48             self._session_memory: Dict[str, List[Dict]] = {}
49             self._init_agents()
50
51         def _init_agents(self):
52             self._agents = {
53                 AgentType.RESEARCHER: ResearcherAgent(),
54                 AgentType.RISK: RiskAgent(),
55                 AgentType.STRATEGIST: StrategistAgent(),
56             }
57             logger.info(f"[Orchestrator] initialized with {len(self._agents)}
agents")
58
59         def _classify_intent(self, query: str) -> str:
60             q = query.lower()
61             if any(k in q for k in ["风险", "风险评估", "暴雷", "违约", "亏损", "预
警"]):
62                 if any(k in q for k in ["全面分析", "深度分析", "完整分析", "投资建
议"]):
63                     return "full_analysis"
64                     return "stock_risk"
65             if any(k in q for k in ["策略", "组合", "配置", "仓位", "建仓", "买入"]):
66                 return "investment_advice"

```

```

67         if any(k in q for k in ["全面分析", "深度分析", "完整分析", "综合分析"]):
68             return "full_analysis"
69         if any(k in q for k in ["报告", "研报", "分析报告"]):
70             return "report"
71         if any(k in q for k in ["分析", "估值", "财务", "营收", "利润", "ROE",
"PE"]):
72             return "stock_analysis"
73         return "general"
74
75     def _extract_stock_codes(self, query: str) -> List[str]:
76         return re.findall(r'\b\d{6}\b', query)
77
78     def _build_context(
79         self, session_id: str, agent_results: Dict[AgentType,
AgentExecutionResult]
80     ) -> Dict[str, Any]:
81         context = {"agent_results": {}, "session_id": session_id,
82                   "timestamp": time.strftime("%Y-%m-%d %H:%M:%S")}
83         for agent_type, result in agent_results.items():
84             if result.success:
85                 context["agent_results"][agent_type.value] = result.output
86         return context
87
88     async def run(
89         self, query: str,
90         session_id: Optional[str] = None,
91         agent_types: Optional[List[AgentType]] = None,
92         parallel: bool = True,
93         context: Optional[Dict[str, Any]] = None,
94     ) -> Dict[str, Any]:
95         session_id = session_id or str(uuid.uuid4())
96         run_id = str(uuid.uuid4())
97
98         logger.info(f"[Orchestrator] run_id={run_id} parallel={parallel}")
99
100         # Step 1: intent classification
101         intent = self._classify_intent(query)
102         stock_codes = self._extract_stock_codes(query)
103
104         # Step 2: select agents
105         if agent_types:
106             selected_agents = agent_types
107         else:
108             selected_agents = [AgentType(a) for a in
self.
109 self.INTENT_AGENT_MAP.get(intent, ["researcher"])]
110
111         # Step 3: execute

```

```

111         agent_results: Dict[AgentType, AgentExecutionResult] = {}
112
113         if parallel and len(selected_agents) > 1:
114             tasks = []
115             for agent_type in selected_agents:
116                 agent = self._agents.get(agent_type)
117                 if agent:
118                     tasks.append((agent_type, self._run_single_agent(agent,
119 query, stock_codes, context)))
120
121             results = await asyncio.gather(*[t for _, t in tasks],
122 return_exceptions=True)
123             for i, (agent_type, _) in enumerate(tasks):
124                 result = results[i]
125                 if isinstance(result, Exception):
126                     agent_results[agent_type] =
127 AgentExecutionResult(success=False, output="", error=str(result))
128                 else:
129                     agent_results[agent_type] = result
130
131         else:
132             shared_context = context or {}
133             for agent_type in selected_agents:
134                 agent = self._agents.get(agent_type)
135                 if agent:
136                     if agent_type == AgentType.RISK and
137 agent_results.get(AgentType.RESEARCHER):
138                         shared_context["researcher_result"] =
139 agent_results[AgentType.RESEARCHER].output
140                     result = await self._run_single_agent(agent, query,
141 stock_codes, shared_context)
142                     agent_results[agent_type] = result
143
144         # Step 4: aggregate
145         final_response = self._aggregate_results(query, agent_results)
146
147         # Step 5: record session
148         self._session_memory.setdefault(session_id, []).append({
149             "run_id": run_id, "query": query, "intent": intent,
150             "agents_used": [a.value for a in selected_agents],
151             "results": {k.value: v.output for k, v in agent_results.items()},
152             "timestamp": time.strftime("%Y-%m-%d %H:%M:%S"),
153         })
154
155         total_time = sum(r.execution_time_ms for r in agent_results.values())
156         total_tokens = sum(r.total_tokens for r in agent_results.values())
157
158         return {

```

```

152         "run_id": run_id, "session_id": session_id, "intent": intent,
153         "agent_results": {k.value: v.output for k, v in
agent_results.items()}},
154         "final_response": final_response,
155         "execution_summary": {
156             "agents_used": [a.value for a in selected_agents],
157             "parallel": parallel, "total_execution_ms": total_time,
158             "total_tokens": total_tokens,
159             "success_count": sum(1 for r in agent_results.values() if
r.success),
160         },
161     }
162
163     async def _run_single_agent(
164         self, agent: Any, query: str, stock_codes: List[str],
165         context: Optional[Dict[str, Any]] = None,
166     ) -> AgentExecutionResult:
167         task = query
168         if stock_codes:
169             task = f"分析股票: {' ', ' '.join(stock_codes)}\n\n用户问题: {query}"
170         try:
171             return await asyncio.wait_for(
172                 agent.execute(task=task, context=context),
173                 timeout=self.config.timeout_per_agent
174             )
175         except asyncio.TimeoutError:
176             return AgentExecutionResult(
177                 success=False, output="", error=f"timeout >
{self.config.timeout_per_agent}s"
178             )
179
180     def _aggregate_results(
181         self, query: str, agent_results: Dict[AgentType, AgentExecutionResult]
182     ) -> str:
183         parts = []
184         for agent_type in [AgentType.RESEARCHER, AgentType.RISK,
AgentType.STRATEGIST]:
185             result = agent_results.get(agent_type)
186             if result and result.success and result.output:
187                 parts.append(f"\n## {agent_type.value.upper()}
Result\n\n{result.output}\n")
188             if not parts:
189                 errors = [f"{k.value}: {v.error}" for k, v in
agent_results.items() if not v.success]
190                 return f"分析遇到问题: {'; '.join(errors)}"
191             return "\n".join(parts)
192

```

```

193     def get_session_memory(self, session_id: str) -> List[Dict]:
194         return self._session_memory.get(session_id, [])
195
196     def clear_session(self, session_id: str):
197         if session_id in self._session_memory:
198             del self._session_memory[session_id]
199
200
201 _orchestrator: Optional[AgentOrchestrator] = None
202
203
204 def get_orchestrator() -> AgentOrchestrator:
205     global _orchestrator
206     if _orchestrator is None:
207         _orchestrator = AgentOrchestrator()
208     return _orchestrator

```

6.7 投研分析 Agent

代码块

```

1  """
2  backend/app/agents/researcher.py
3  投研分析 Agent - 股票基本面分析、财务解读
4  """
5  from typing import Dict, Any
6  from app.agents.base import BaseAgent, AgentConfig, AgentExecutionResult
7  from app.models.schemas import AgentType
8  from langchain.tools import tool
9
10
11 @tool(description="查询股票基本信息（代码、名称、行业、上市时间、市值等）")
12 def get_stock_info(stock_code: str) -> str:
13     return f'{{"code": "{stock_code}", "name": "股票名称", "industry": "行业"}}'
14
15
16 @tool(description="查询股票财务数据（营收、净利润、毛利率、ROE等）")
17 def get_stock_financials(stock_code: str, period: str = "annual") -> str:
18     return f'{{"code": "{stock_code}", "period": "{period}", "revenue": 0, "net_profit": 0, "roe": 0.0}}'
19
20
21 @tool(description="查询股票K线数据（OHLCV）")
22 def get_stock_kline(stock_code: str, days: int = 60) -> str:
23     return f'[[{"date": "2024-01-01", "open": 0, "high": 0, "low": 0, "close": 0, "volume": 0}]]'

```

```

24
25
26 @tool(description="查询财经新闻")
27 def search_financial_news(keyword: str, limit: int = 10) -> str:
28     return f'[{"title": "新闻标题", "source": "来源", "date": "2024-01-01"}]'
29
30
31 RESEARCHER_SYSTEM_PROMPT = """你是资深金融投研分析师，10年+经验，擅长基本面分析、财务
建模。
32
33 分析框架：
34 - **业绩驱动因素**：营收增长来源、市场份额变化
35 - **盈利能力分析**：毛利率、净利率、ROE 趋势
36 - **估值判断**：PE/PB/PS 横向（vs 行业）和纵向（vs 历史）对比
37 - **风险提示**：经营风险、财务风险、行业周期性
38 - **投资建议**：客观结论
39
40 输出：Markdown结构，数据有来源，关键结论**加粗**，风险⚠️开头"""
41
42
43 class ResearcherAgent(BaseAgent):
44     def __init__(self):
45         tools = [get_stock_info, get_stock_financials, get_stock_kline,
search_financial_news]
46         config = AgentConfig(
47             name="投研分析Agent", agent_type=AgentType.RESEARCHER,
48             description="股票基本面分析、财务解读",
49             system_prompt=RESEARCHER_SYSTEM_PROMPT, tools=tools,
max_iterations=8,
50         )
51         super().__init__(config)
52
53     def _build_system_prompt(self) -> str:
54         return self.config.system_prompt
55
56     def _parse_output(self, raw_output: str) -> Dict[str, Any]:
57         return {"analysis": raw_output, "agent_type": "researcher"}
58
59     async def analyze_stock(self, stock_code: str, query: str = "") ->
AgentExecutionResult:
60         task = f"分析股票 {stock_code}。获取基本信息、财务数据，进行估值分析，对比行
业，输出综合研判。{query}"
61         return await self.execute(task=task)

```

6.8 风控预警 Agent

```

1 代码块"""
2  backend/app/agents/risk_agent.py
3  风控预警 Agent - 风险识别、量化评估、预警
4  """
5  from typing import Dict, Any
6  from app.agents.base import BaseAgent, AgentConfig, AgentExecutionResult
7  from app.models.schemas import AgentType
8  from langchain.tools import tool
9
10
11 @tool(description="计算股票风险指标（波动率、VaR、Beta等）")
12 def calculate_risk_metrics(stock_code: str, period: int = 60) -> str:
13     return '{"code": "stock_code", "volatility": 0.25, "var_95": -0.03, "beta": 1.2}'
14
15
16 @tool(description="检查监管处罚和诉讼信息")
17 def check_regulatory_events(stock_code: str) -> str:
18     return '[{"event": "处罚/诉讼事件", "date": "日期", "severity": "严重程度"}]'
19
20
21 RISK_AGENT_SYSTEM_PROMPT = """你是专业金融风控专家，擅长风险量化、预警模型、合规审查。
22
23 风控框架：
24 - 财务风险：盈利质量、现金流异常、债务结构
25 - 经营风险：大客户依赖、竞争壁垒、管理层变更
26 - 市场风险：波动性、VaR、Beta、系统性风险
27 - 合规风险：监管处罚、信息披露违规
28
29 风险评分：
30 - 红色 高风险（80-100）
31 - 橙色 中高风险（60-79）
32 - 黄色 中风险（40-59）
33 - 绿色 低风险（0-39）"""
34
35
36 class RiskAgent(BaseAgent):
37     def __init__(self):
38         tools = [calculate_risk_metrics, check_regulatory_events]
39         config = AgentConfig(
40             name="风控预警Agent", agent_type=AgentType.RISK,
41             description="风险识别、量化评估、预警监控",
42             system_prompt=RISK_AGENT_SYSTEM_PROMPT, tools=tools,
43             max_iterations=6,
44             )
45         super().__init__(config)

```

```

46     def _build_system_prompt(self) -> str:
47         return self.config.system_prompt
48
49     def _parse_output(self, raw_output: str) -> Dict[str, Any]:
50         return {"analysis": raw_output, "agent_type": "risk"}
51
52     async def assess_risk(self, stock_code: str, query: str = "") ->
AgentExecutionResult:
53         task = f"评估股票 {stock_code} 风险。计算风险指标，检查监管事件，识别财务风
险，输出风险评分（0-100分）。{query}"
54         return await self.execute(task=task)

```

6.9 策略生成

代码块

```

1     """
2     backend/app/agents/strategist.py
3     策略生成 Agent - 投资策略、组合建议、交易方案
4     """
5     from typing import Dict, Any
6     from app.agents.base import BaseAgent, AgentConfig, AgentExecutionResult
7     from app.models.schemas import AgentType
8
9
10    STRATEGIST_SYSTEM_PROMPT = """你是资深量化投资策略师，精通多因子模型、资产配置。
11
12    策略原则：
13    - 风险收益匹配：诚实告知用户
14    - 分散化：单一标的仓位不超过20%
15    - 流动性优先：日均成交额>1亿
16    - 动态调整：根据市场环境调整
17
18    输出框架：
19    1. 策略概述：类型、适用场景、预期收益与风险
20    2. 选股标准：行业、市值、估值筛选条件
21    3. 仓位配置：各标的仓位、入场点位、止损线
22    4. 风险控制：最大回撤控制、动态止损规则
23
24    重要声明：仅供参考，不构成投资建议"""
25
26
27    class StrategistAgent(BaseAgent):
28        def __init__(self):
29            config = AgentConfig(
30                name="策略生成Agent", agent_type=AgentType.STRATEGIST,

```

```

31         description="投资策略、组合建议、交易方案",
32         system_prompt=STRATEGIST_SYSTEM_PROMPT, tools=[], max_iterations=5,
33     )
34     super().__init__(config)
35
36     def _build_system_prompt(self) -> str:
37         return self.config.system_prompt
38
39     def _parse_output(self, raw_output: str) -> Dict[str, Any]:
40         return {"strategy": raw_output, "agent_type": "strategist"}
41
42     async def generate_strategy(self, context: Dict[str, Any]) ->
AgentExecutionResult:
43         task = f"""基于以下分析结果生成投资策略：
44
45 研究结论：{context.get('researcher_result', '无')}
46 风控结论：{context.get('risk_result', '无')}
47 用户偏好：{context.get('user_preference', '无')}
48
49 请按策略框架输出。"""
50         return await self.execute(task=task, context=context)

```

6.10 报告生成

代码块

```

1  """
2  backend/app/agents/report_agent.py
3  报告生成 Agent - 自动生成结构化研报报告
4  """
5  from typing import Dict, Any
6  from app.agents.base import BaseAgent, AgentConfig, AgentExecutionResult
7  from app.models.schemas import AgentType
8
9
10 REPORT_SYSTEM_PROMPT = """你是专业金融研究报告撰写专家。
11
12 报告框架：
13  1. 执行摘要（100字内）
14  2. 公司概况：主营业务、市场地位
15  3. 基本面分析：营收、利润、现金流趋势
16  4. 估值分析：PE/PB/PS 对比
17  5. 风险因素
18  6. 投资建议（买入/持有/卖出）
19
20 格式：Markdown，数据表格，结论加粗"""

```

```

21
22
23 class ReportAgent(BaseAgent):
24     def __init__(self):
25         config = AgentConfig(
26             name="报告生成Agent", agent_type=AgentType.REPORT,
27             description="自动生成结构化研报报告",
28             system_prompt=REPORT_SYSTEM_PROMPT, tools=[], max_iterations=3,
29         )
30         super().__init__(config)
31
32     def _build_system_prompt(self) -> str:
33         return self.config.system_prompt
34
35     def _parse_output(self, raw_output: str) -> Dict[str, Any]:
36         return {"report": raw_output, "agent_type": "report"}
37
38     async def generate_report(self, stock_code: str, analysis_data: Dict[str,
Any]) -> AgentExecutionResult:
39         task = f"为股票 {stock_code} 生成完整研报报告。数据: {analysis_data}"
40         return await self.execute(task=task)

```

6.11 RAG 知识库管理

代码块

```

1  """
2  backend/app/rag/knowledge_base.py
3  RAG 知识库管理 - 文档加载、分块、索引
4  """
5  import uuid
6  import hashlib
7  import logging
8  from typing import List, Dict, Any, Optional
9  from dataclasses import dataclass
10
11  from langchain.text_splitter import RecursiveCharacterTextSplitter
12  from langchain.schema import Document
13
14  from app.core.llm import get_llm_service, LLMService
15  from app.config import get_settings
16
17  logger = logging.getLogger(__name__)
18  settings = get_settings()
19
20

```

```

21 @dataclass
22 class ChunkResult:
23     chunk_id: str
24     content: str
25     metadata: Dict[str, Any]
26     embedding: Optional[List[float]] = None
27
28
29 class KnowledgeBaseManager:
30     """
31     知识库管理器
32     职责：文档加载 | 文档分块 | 向量化存储 | 知识检索
33     """
34
35     def __init__(self, llm_service: Optional[LLMService] = None):
36         self.llm = llm_service or get_llm_service()
37         self.settings = get_settings()
38         self.text_splitter = RecursiveCharacterTextSplitter(
39             chunk_size=500, chunk_overlap=50,
40             length_function=len,
41             separators=["\n\n", "\n", ". ", "! ", "? ", "; ", ", ", " ", ""],
42         )
43         self._collection = None
44
45     @property
46     def collection(self):
47         if self._collection is None:
48             try:
49                 import chromadb
50                 from chromadb.config import Settings as ChromaSettings
51                 client = chromadb.PersistentClient(
52                     path=self.settings.CHROMA_PERSIST_DIR,
53                     settings=ChromaSettings(anonymized_telemetry=False)
54                 )
55                 self._collection = client.get_or_create_collection(
56                     name=self.settings.CHROMA_COLLECTION_NAME,
57                     metadata={"hnsw:space": "cosine"}
58                 )
59                 logger.info(f"[KB] ChromaDB initialized:
60 {self.settings.CHROMA_COLLECTION_NAME}")
61             except ImportError:
62                 logger.warning("[KB] ChromaDB not installed")
63                 self._collection = None
64             return self._collection
65
66     def chunk_documents(
67         self, documents: List[Document],

```

```

67         chunk_size: int = 500, chunk_overlap: int = 50,
68     ) -> List[ChunkResult]:
69         """文档分块"""
70         self.text_splitter.chunk_size = chunk_size
71         self.text_splitter.chunk_overlap = chunk_overlap
72         chunks = self.text_splitter.split_documents(documents)
73         results = []
74         for i, chunk in enumerate(chunks):
75             chunk_id = str(uuid.uuid4())
76             results.append(ChunkResult(
77                 chunk_id=chunk_id,
78                 content=chunk.page_content,
79                 metadata={
80                     **chunk.metadata,
81                     "chunk_index": i,
82                     "total_chunks": len(chunks),
83                     "content_hash":
hashlib.md5(chunk.page_content.encode()).hexdigest(),
84                 }
85             ))
86         logger.info(f"[KB] chunked: {len(documents)} docs -> {len(results)}
chunks")
87         return results
88
89     async def add_chunks_to_vectorstore(
90         self, chunks: List[ChunkResult], batch_size: int = 100,
91     ) -> int:
92         """将文档块添加到向量存储"""
93         if not self.collection:
94             logger.warning("[KB] vectorstore not initialized")
95             return 0
96
97         texts = [c.content for c in chunks]
98         metadatas = [c.metadata for c in chunks]
99         ids = [c.chunk_id for c in chunks]
100
101         total_added = 0
102         for i in range(0, len(texts), batch_size):
103             batch_texts = texts[i:i + batch_size]
104             batch metas = metadatas[i:i + batch_size]
105             batch_ids = ids[i:i + batch_size]
106             embeddings = await self.llm.embed(batch_texts)
107             self.collection.add(embeddings=embeddings, documents=batch_texts,
metadatas=batch metas, ids=batch_ids)
108             total_added += len(batch_texts)
109
110         logger.info(f"[KB] added {total_added} chunks to vectorstore")

```

```

111         return total_added
112
113     async def add_text_chunks(
114         self, texts: List[str], category: str = "general",
115         metadata_list: Optional[List[Dict]] = None,
116     ) -> int:
117         """直接添加文本块"""
118         results = []
119         for i, text in enumerate(texts):
120             meta = metadata_list[i] if metadata_list else {}
121             results.append(ChunkResult(
122                 chunk_id=str(uuid.uuid4()), content=text,
123                 metadata={**meta, "category": category}
124             ))
125         return await self.add_chunks_to_vectorstore(results)
126
127     def count_documents(self) -> int:
128         if self.collection:
129             return self.collection.count()
130         return 0
131
132
133 kb_manager = KnowledgeBaseManager()
134
135
136 def get_kb_manager() -> KnowledgeBaseManager:
137     return kb_manager

```

6.12 RAG 混合检索器

代码块

```

1     """
2     backend/app/rag/retriever.py
3     混合检索器 - Sparse + Dense 融合检索 + Rerank
4     """
5     import logging
6     from typing import List, Dict, Any, Optional
7
8     from app.core.llm import get_llm_service, LLMService
9     from app.rag.knowledge_base import get_kb_manager, KnowledgeBaseManager
10
11     logger = logging.getLogger(__name__)
12
13
14     class HybridRetriever:

```

```

15     """
16     混合检索器
17     流程: Query Rewrite -> Sparse检索 -> Dense检索 -> RRF融合 -> Rerank -> GRAG
18     """
19
20     def __init__(self, llm_service: Optional[LLMService] = None):
21         self.llm = llm_service or get_llm_service()
22         self.kb = get_kb_manager()
23
24     async def retrieve(
25         self, query: str,
26         top_k: int = 10,
27         score_threshold: float = 0.5,
28         filters: Optional[Dict[str, Any]] = None,
29         enable_rerank: bool = True,
30     ) -> List[Dict[str, Any]]:
31         """
32         执行混合检索
33
34         Args:
35             query: 用户查询
36             top_k: 返回数量
37             score_threshold: 相似度阈值
38             filters: 元数据过滤
39             enable_rerank: 是否启用重排序
40         """
41         logger.info(f"[Retriever] query={query[:50]} top_k={top_k} rerank={enable_rerank}")
42
43         # Step 1: 向量化查询
44         query_embedding = await self.llm.embed([query])
45         if not query_embedding:
46             return []
47         query_vec = query_embedding[0]
48
49         # Step 2: ChromaDB 向量检索
50         collection = self.kb.collection
51         if not collection:
52             logger.warning("[Retriever] vectorstore empty")
53             return []
54
55         where_filter = filters
56         try:
57             results = collection.query(
58                 query_embeddings=[query_vec],
59                 n_results=top_k * 2,
60                 where=where_filter,

```

```

61         include=["documents", "metadatas", "distances"],
62     )
63
64     if not results or not results.get("ids"):
65         return []
66
67     ids = results["ids"][0]
68     documents = results["documents"][0]
69     metadatas = results.get("metadatas", [[]])[0]
70     distances = results.get("distances", [[]])[0]
71
72     # cosine distance -> similarity
73     scores = [1 - d for d in distances]
74
75     chunks = []
76     for i in range(len(ids)):
77         chunks.append({
78             "chunk_id": ids[i],
79             "content": documents[i],
80             "score": scores[i] if i < len(scores) else 0.0,
81             "metadata": metadatas[i] if i < len(metadatas) else {},
82         })
83
84     except Exception as e:
85         logger.error(f"[Retriever] ChromaDB search failed: {str(e)}")
86         return []
87
88     # Step 3: 过滤低分
89     chunks = [c for c in chunks if c["score"] >= score_threshold]
90
91     # Step 4: 重排序
92     if enable_rerank and len(chunks) > 1:
93         chunks = await self._rerank(query, chunks, top_k=top_k)
94
95     # Step 5: top_k
96     chunks = chunks[:top_k]
97     logger.info(f"[Retriever] returned {len(chunks)} results")
98     return chunks
99
100     async def _rerank(
101         self, query: str, chunks: List[Dict[str, Any]], top_k: int = 5
102     ) -> List[Dict[str, Any]]:
103         """Cross-Encoder 重排序"""
104         try:
105             from sentence_transformers import CrossEncoder
106             model = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2",
max_length=512)

```

```

107         pairs = [(query, chunk["content"]) for chunk in chunks]
108         scores = model.predict(pairs)
109         for i, chunk in enumerate(chunks):
110             chunk["rerank_score"] = float(scores[i])
111             chunk["score"] = float(scores[i])
112         chunks.sort(key=lambda x: x["rerank_score"], reverse=True)
113         logger.info("[Retriever] rerank applied with Cross-Encoder")
114     except Exception as e:
115         logger.warning(f"[Retriever] rerank failed: {str(e)}")
116     return chunks
117
118     async def retrieve_with_expansion(
119         self, query: str, top_k: int = 10, **kwargs
120     ) -> List[Dict[str, Any]]:
121         """带查询扩展的检索"""
122         try:
123             from langchain.schema import HumanMessage
124             llm = get_llm_service()
125             expansion_prompt = f"为以下金融查询生成3个同义表达，原始查询：{query}"
126             messages = [HumanMessage(content=expansion_prompt)]
127             log = await llm.chat(messages, stream=False)
128             expanded = [query]
129             all_chunks = {}
130             for q in expanded:
131                 chunks = await self.retrieve(q, top_k=top_k // 2, **kwargs)
132                 for c in chunks:
133                     cid = c["chunk_id"]
134                     if cid not in all_chunks or c["score"] > all_chunks[cid]
135                         ["score"]:
136                             all_chunks[cid] = c
137             merged = sorted(all_chunks.values(), key=lambda x: x["score"],
138                             reverse=True)
139             return merged[:top_k]
140         except Exception as e:
141             logger.warning(f"[Retriever] expansion failed: {str(e)}")
142             return await self.retrieve(query, top_k, **kwargs)
143
144     hybrid_retriever = HybridRetriever()
145
146     def get_hybrid_retriever() -> HybridRetriever:
147         return hybrid_retriever

```

6.13 查询改写模块

```

1  """
2  backend/app/rag/query_rewrite.py
3  查询改写模块 - 通过 LLM 优化用户查询
4  """
5  import re
6  import json
7  import logging
8  from typing import Dict, Any
9  from app.core.llm import get_llm_service
10
11  logger = logging.getLogger(__name__)
12
13
14  class QueryRewriter:
15      """
16      查询改写器
17      功能：意图澄清 | 同义词扩展 | 查询分解 | 格式修正
18      """
19
20      REWRITE_PROMPT = """你是专业金融搜索查询优化助手。
21  原始查询：{original_query}
22
23  请优化查询：
24  1. 术语标准化："茅台" -> "贵州茅台(600519)"
25  2. 意图识别：投研分析 / 风险评估 / 策略建议 / 知识问答
26  3. 查询扩展（可选）：补充同义词
27
28  JSON格式输出：
29  {"rewritten_query": "优化后查询", "intent": "意图", "stock_codes": [],
30   "stock_names": [], "notes": ""}
31  只输出JSON，不要其他内容。"""
32
33      def __init__(self):
34          self.llm = get_llm_service()
35
36      async def rewrite(self, query: str) -> Dict[str, Any]:
37          try:
38              from langchain.schema import HumanMessage
39              messages =
40              [HumanMessage(content=self.REWRITE_PROMPT.format(original_query=query))]
41              log = await self.llm.chat(messages, stream=False)
42              result_text = log.error or "{}"
43              json_match = re.search(r'\{(?:\[.*\]|(?:[^\[\]]|\[.*\])*)*\}',
44              result_text, re.DOTALL)
45              if json_match:
46                  return json.loads(json_match.group(0))

```

```

44         return {"rewritten_query": query, "intent": "unknown",
    "stock_codes": [], "stock_names": [], "expand_queries": []}
45     except Exception as e:
46         logger.warning(f"[QueryRewrite] failed: {str(e)}")
47         return {"rewritten_query": query, "intent": "unknown",
    "stock_codes": [], "stock_names": [], "expand_queries": []}

```

6.14 LLMOps 监控模块

代码块

```

1  """
2  backend/app/llmops/monitor.py
3  LLMOps 监控模块 - 调用日志、成本追踪、指标暴露
4  """
5  import time
6  import uuid
7  import logging
8  from typing import Dict, Any, List, Optional
9  from datetime import datetime, timedelta
10 from collections import defaultdict
11 from dataclasses import dataclass
12 from threading import Lock
13
14 logger = logging.getLogger(__name__)
15
16
17 @dataclass
18 class LLMCallRecord:
19     call_id: str
20     model: str
21     prompt_tokens: int
22     completion_tokens: int
23     total_tokens: int
24     latency_ms: float
25     cost_usd: float
26     success: bool
27     error: Optional[str] = None
28     session_id: Optional[str] = None
29     prompt_preview: str = ""
30     timestamp: str = ""
31
32
33 class LLMOpsMonitor:
34     """
35     LLMOps 监控器

```

```

36 功能：全量记录LLM调用日志 | 实时计算Token消耗和成本
37 统计成功率/平均延迟 | Prometheus指标暴露
38 """
39
40 def __init__(self):
41     self._records: List[LLMCallRecord] = []
42     self._daily_stats: Dict[str, Dict] = defaultdict(lambda: {
43         "total_calls": 0, "total_tokens": 0, "total_cost": 0.0,
44         "total_latency": 0.0, "success_calls": 0, "fail_calls": 0,
45     })
46     self._lock = Lock()
47     self._max_records = 10000
48     logger.info("[LLMOps] monitor initialized")
49
50 def log_call(
51     self, model: str, prompt_tokens: int, completion_tokens: int,
52     total_tokens: int, latency_ms: float, cost_usd: float,
53     success: bool, error: Optional[str] = None,
54     session_id: Optional[str] = None,
55     prompt_preview: str = "",
56 ) -> str:
57     call_id = str(uuid.uuid4())
58     timestamp = time.strftime("%Y-%m-%d %H:%M:%S")
59     today = timestamp[:10]
60
61     record = LLMCallRecord(
62         call_id=call_id, model=model, prompt_tokens=prompt_tokens,
63         completion_tokens=completion_tokens, total_tokens=total_tokens,
64         latency_ms=latency_ms, cost_usd=cost_usd, success=success,
65         error=error, session_id=session_id,
66         prompt_preview=prompt_preview[:100],
67         timestamp=timestamp,
68     )
69
70     with self._lock:
71         self._records.append(record)
72         if len(self._records) > self._max_records:
73             self._records = self._records[-self._max_records // 2:]
74
75         stats = self._daily_stats[today]
76         stats["total_calls"] += 1
77         stats["total_tokens"] += total_tokens
78         stats["total_cost"] += cost_usd
79         stats["total_latency"] += latency_ms
80         if success:
81             stats["success_calls"] += 1
82         else:

```

```

82         stats["fail_calls"] += 1
83
84     logger.debug(f"[LLMOps] log_call {model} tokens={total_tokens}")
85     return call_id
86
87     def get_metrics(self, days: int = 7) -> Dict[str, Any]:
88         with self._lock:
89             today = time.strftime("%Y-%m-%d")
90             today_stats = self._daily_stats.get(today, {})
91
92             totals = {"total_calls": 0, "total_tokens": 0, "total_cost": 0.0,
93                      "total_latency": 0.0, "success_calls": 0, "fail_calls":
0}
94
95             for d in range(days):
96                 day = (datetime.now() - timedelta(days=d)).strftime("%Y-%m-%d")
97                 stats = self._daily_stats.get(day, {})
98                 for k in totals:
99                     totals[k] += stats.get(k, 0)
100
101             total_calls = totals["total_calls"] or 1
102             success_rate = totals["success_calls"] / total_calls
103             today_calls = today_stats.get("total_calls", 0) or 1
104             today_success_rate = today_stats.get("success_calls", 0) /
today_calls
105
106             return {
107                 "all_time": {
108                     "total_calls": totals["total_calls"],
109                     "total_tokens": totals["total_tokens"],
110                     "total_cost_usd": round(totals["total_cost"], 6),
111                     "avg_latency_ms": round(totals["total_latency"] /
total_calls, 2),
112                     "success_rate": round(success_rate * 100, 2),
113                 },
114                 "today": {
115                     "calls": today_stats.get("total_calls", 0),
116                     "tokens": today_stats.get("total_tokens", 0),
117                     "cost_usd": round(today_stats.get("total_cost", 0), 6),
118                     "avg_latency_ms": round(today_stats.get("total_latency",
0) / today_calls, 2),
119                     "success_rate": round(today_success_rate * 100, 2),
120                 },
121             }
122
123     def get_daily_stats(self, days: int = 30) -> List[Dict]:
124         result = []
125         with self._lock:

```

```

125         for d in range(days):
126             day = (datetime.now() - timedelta(days=d)).strftime("%Y-%m-%d")
127             stats = self._daily_stats.get(day, {})
128             total_calls = stats.get("total_calls", 0) or 1
129             result.append({
130                 "date": day,
131                 "calls": stats.get("total_calls", 0),
132                 "tokens": stats.get("total_tokens", 0),
133                 "cost_usd": round(stats.get("total_cost", 0), 6),
134                 "avg_latency_ms": round(stats.get("total_latency", 0) /
total_calls, 2),
135                 "success_rate": round(stats.get("success_calls", 0) /
total_calls * 100, 2),
136             })
137         return result[::-1]
138
139     def get_recent_calls(self, limit: int = 50) -> List[Dict]:
140         with self._lock:
141             records = self._records[-limit:][::-1]
142             return [{
143                 "call_id": r.call_id, "model": r.model, "tokens":
r.total_tokens,
144                 "latency_ms": round(r.latency_ms, 2), "cost_usd":
round(r.cost_usd, 6),
145                 "success": r.success, "error": r.error, "timestamp":
r.timestamp,
146             } for r in records]
147
148
149 llmops_monitor = LLMOpsMonitor()
150
151
152 def get_llmops_monitor() -> LLMOpsMonitor:
153     return llmops_monitor

```

6.15 DataOps 数据管道编排

代码块

```

1  """
2  backend/app/dataops/pipeline.py
3  DataOps 数据管道编排 - 金融数据 ETL
4  """
5  import logging
6  from typing import Dict, Any, List, Optional, Callable
7  from datetime import datetime

```

```

8  from dataclasses import dataclass
9  import asyncio
10
11  logger = logging.getLogger(__name__)
12
13
14  @dataclass
15  class PipelineStage:
16      name: str
17      description: str
18      extractor: Optional[Callable] = None
19      transformer: Optional[Callable] = None
20      loader: Optional[Callable] = None
21      validator: Optional[Callable] = None
22
23
24  class DataPipeline:
25      """
26      金融数据 ETL 管道
27      阶段: Extract -> Transform -> Validate -> Load
28      """
29
30      def __init__(self, name: str):
31          self.name = name
32          self.stages: List[PipelineStage] = []
33          self._is_running = False
34          self._last_run_at: Optional[datetime] = None
35          self._last_status: str = "idle"
36
37      def add_stage(self, stage: PipelineStage):
38          self.stages.append(stage)
39          return self
40
41      async def run(self, params: Optional[Dict[str, Any]] = None) -> Dict[str,
Any]:
42          import uuid
43          run_id = str(uuid.uuid4())
44          params = params or {}
45          start_time = datetime.now()
46
47          logger.info(f"[Pipeline:{self.name}] starting run_id={run_id}")
48          self._is_running = True
49
50          total_records = 0
51          stage_results = {}
52
53          try:

```

```

54         for stage in self.stages:
55             logger.info(f"[Pipeline:{self.name}] stage: {stage.name}")
56
57             extracted_data = []
58             if stage.extractor:
59                 extracted_data = await self._safe_call(stage.extractor,
60 params)
61
62             transformed_data = []
63             if stage.transformer:
64                 transformed_data = await
65 self._safe_call(stage.transformer, extracted_data, params)
66
67             validation_result = {"passed": True, "errors": []}
68             if stage.validator:
69                 validation_result = await self._safe_call(stage.validator,
70 transformed_data)
71
72             loaded_count = 0
73             if stage.loader:
74                 loaded_count = await self._safe_call(stage.loader,
75 transformed_data)
76
77             total_records += loaded_count
78
79             stage_results[stage.name] = {
80                 "extracted": len(extracted_data),
81                 "transformed": len(transformed_data),
82                 "loaded": loaded_count,
83                 "validated": validation_result.get("passed", False),
84             }
85
86             self._last_status = "success"
87             self._last_run_at = datetime.now()
88
89             return {
90                 "run_id": run_id, "pipeline": self.name, "status": "success",
91                 "total_records": total_records, "stage_results": stage_results,
92                 "started_at": start_time.isoformat(),
93                 "completed_at": datetime.now().isoformat(),
94             }
95
96         except Exception as e:
97             logger.error(f"[Pipeline:{self.name}] failed: {str(e)}",
98 exc_info=True)
99
100             self._last_status = "failed"
101             return {
102                 "run_id": run_id, "pipeline": self.name, "status": "failed",

```

```

96         "error": str(e), "total_records": total_records,
97         "stage_results": stage_results,
98         "started_at": start_time.isoformat(),
99         "completed_at": datetime.now().isoformat(),
100     }
101     finally:
102         self._is_running = False
103
104     async def _safe_call(self, func: Callable, *args, **kwargs) -> Any:
105         try:
106             if asyncio.iscoroutinefunction(func):
107                 return await func(*args, **kwargs)
108             return func(*args, **kwargs)
109         except Exception as e:
110             logger.error(f"[Pipeline] stage failed: {str(e)}")
111             return []
112
113
114     # ===== 预置抽取器 =====
115
116     async def wind_stock_quote_extractor(params: Dict) -> List[Dict]:
117         """Wind 股票行情抽取器"""
118         return [{"stock_code": "000001", "stock_name": "平安银行",
119                 "price": 12.50, "change_pct": 1.23, "volume": 50000000,
120                 "timestamp": datetime.now().isoformat()}]
121
122
123     async def tushare_financial_extractor(params: Dict) -> List[Dict]:
124         """Tushare 财务数据抽取器"""
125         return [{"stock_code": "600519", "report_date": "2024-06-30",
126                 "revenue": 83000000000, "net_profit": 42000000000,
127                 "gross_margin": 0.92, "roe": 0.15}]
128
129
130     async def stock_quote_transformer(data: List[Dict], params: Dict) ->
List[Dict]:
131         """行情数据转换"""
132         return [{"code": item.get("stock_code", ""),
133                 "name": item.get("stock_name", ""),
134                 "price": float(item.get("price", 0)),
135                 "change_pct": float(item.get("change_pct", 0)),
136                 "volume": int(item.get("volume", 0))} for item in data]
137
138
139     async def data_quality_validator(data: List[Dict]) -> Dict:
140         """数据质量校验"""
141         errors = []

```

```

142     for i, item in enumerate(data[:10]):
143         if not item.get("code"):
144             errors.append(f"第{i}条缺少股票代码")
145     return {"passed": len(errors) == 0, "errors": errors, "checked": len(data)}
146
147
148 async def db_loader(data: List[Dict]) -> int:
149     """数据库加载"""
150     logger.info(f"[DBLoader] loading {len(data)} records")
151     return len(data)

```

6.16 FastAPI 应用入口

代码块

```

1  """
2  backend/app/main.py
3  FastAPI 应用入口
4  """
5  import logging
6  from contextlib import asynccontextmanager
7
8  from fastapi import FastAPI, Request
9  from fastapi.middleware.cors import CORSMiddleware
10 from fastapi.middleware.gzip import GZipMiddleware
11 from fastapi.responses import JSONResponse
12
13 from app.config import get_settings
14 from app.api import chat, agent, rag, llmops, data
15
16 logging.basicConfig(
17     level=logging.INFO,
18     format="%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",
19     datefmt="%Y-%m-%d %H:%M:%S",
20 )
21 logger = logging.getLogger(__name__)
22 settings = get_settings()
23
24
25 @asynccontextmanager
26 async def lifespan(app: FastAPI):
27     logger.info(f"Financial AI Agent v{settings.APP_VERSION} starting...")
28     logger.info(f"LLM: {settings.LLM_MODEL}, VectorStore:
29 {settings.VECTORSTORE_TYPE}")
30     yield
31     logger.info("Financial AI Agent shutting down...")

```

```
31
32
33 app = FastAPI(
34     title=settings.APP_NAME,
35     version=settings.APP_VERSION,
36     description="基于 LLM + Agent + RAG 的智能金融分析平台 API",
37     lifespan=lifespan,
38     docs_url="/docs",
39     redoc_url="/redoc",
40 )
41
42 app.add_middleware(GZipMiddleware, minimum_size=1000)
43 app.add_middleware(
44     CORSMiddleware,
45     allow_origins=settings.CORS_ORIGINS,
46     allow_credentials=True,
47     allow_methods=["*"],
48     allow_headers=["*"],
49 )
50
51
52 @app.exception_handler(Exception)
53 async def global_exception_handler(request: Request, exc: Exception):
54     logger.error(f"[GlobalException] {str(exc)}", exc_info=True)
55     return JSONResponse(status_code=500, content={"detail": str(exc)})
56
57
58 @app.get("/health")
59 async def health_check():
60     return {"status": "healthy", "app": settings.APP_NAME, "version":
settings.APP_VERSION,
61           "llm_model": settings.LLM_MODEL}
62
63
64 @app.get("/")
65 async def root():
66     return {"app": settings.APP_NAME, "version": settings.APP_VERSION, "docs":
"/docs"}
67
68
69 # 注册路由
70 app.include_router(chat.router, prefix="/api/v1", tags=["对话"])
71 app.include_router(agent.router, prefix="/api/v1", tags=["Agent编排"])
72 app.include_router(rag.router, prefix="/api/v1", tags=["RAG知识库"])
73 app.include_router(llmops.router, prefix="/api/v1", tags=["LLMOps监控"])
74 app.include_router(data.router, prefix="/api/v1", tags=["金融数据"])
75
```

```

76
77 if __name__ == "__main__":
78     import uvicorn
79     uvicorn.run("app.main:app", host=settings.HOST, port=settings.PORT,
80                 reload=settings.DEBUG, workers=1, log_level="info")

```

6.17 对话聊天 API

代码块

```

1  """
2  backend/app/api/chat.py
3  对话聊天 API
4  """
5  import uuid
6  import time
7  import logging
8  from fastapi import APIRouter, HTTPException
9  from fastapi.responses import StreamingResponse
10 from app.models.schemas import ChatRequest, ChatResponse, AgentType
11 from app.agents.orchestrator import get_orchestrator
12 from app.llmops.monitor import get_llmops_monitor
13 from app.rag.retriever import get_hybrid_retriever
14
15 logger = logging.getLogger(__name__)
16 router = APIRouter()
17
18
19 @router.post("/chat", response_model=ChatResponse)
20 async def chat(request: ChatRequest):
21     """
22     对话接口
23     流程: 接收消息 -> RAG检索 -> Agent执行 -> 返回结果
24     """
25     session_id = request.session_id or str(uuid.uuid4())
26     start_time = time.time()
27     context = {}
28
29     try:
30         # RAG 检索
31         sources = []
32         if request.enable_rag:
33             retriever = get_hybrid_retriever()
34             chunks = await retriever.retrieve(query=request.message, top_k=5,
enable_rerank=True)
35             if chunks:

```

```

36         context["rag_context"] = chunks
37         sources = [{"content": c["content"][:200] + "...", "score":
round(c["score"], 3),
38                     "metadata": c.get("metadata", {})} for c in chunks]
39
40     # Agent 执行
41     orchestrator = get_orchestrator()
42     agent_types = [request.agent_type] if request.agent_type else None
43     result = await orchestrator.run(
44         query=request.message, session_id=session_id,
45         agent_types=agent_types, parallel=False, context=context,
46     )
47
48     latency = (time.time() - start_time) * 1000
49
50     # 记录 LLMops
51     monitor = get_llmops_monitor()
52     monitor.log_call(
53         model="orchestrator", prompt_tokens=0,
54         completion_tokens=len(result["final_response"]) // 4,
55         total_tokens=result["execution_summary"]["total_tokens"],
56         latency_ms=latency, cost_usd=0.0, success=True,
57         session_id=session_id, prompt_preview=request.message[:100],
58     )
59
60     return ChatResponse(
61         session_id=session_id, message=result["final_response"],
62         agent_type=AgentType.RESEARCHER,
63         sources=sources if sources else None,
64         token_usage={"total": result["execution_summary"]["total_tokens"]},
65         latency_ms=round(latency, 2),
66     )
67
68     except Exception as e:
69         logger.error(f"[Chat] failed: {str(e)}")
70         raise HTTPException(status_code=500, detail=str(e))
71
72
73 @router.post("/chat/stream")
74 async def chat_stream(request: ChatRequest):
75     """流式对话接口 (SSE) """
76     session_id = request.session_id or str(uuid.uuid4())
77
78     async def generate():
79         try:
80             orchestrator = get_orchestrator()

```

```

81         result = await orchestrator.run(query=request.message,
session_id=session_id)
82         response_text = result["final_response"]
83         for i in range(0, len(response_text), 50):
84             chunk = response_text[i:i + 50]
85             yield f"event: message\ndata: {chunk}\n\n"
86             yield f"event: done\ndata: done\n\n"
87         except Exception as e:
88             yield f"event: error\ndata: {str(e)}\n\n"
89
90     return StreamingResponse(generate(), media_type="text/event-stream",
91                             headers={"Cache-Control": "no-cache", "X-Accel-
Buffering": "no"})

```

6.18 Agent 编排 API

代码块

```

1  """
2  backend/app/api/agent.py
3  Agent 编排 API
4  """
5  import uuid
6  import logging
7  from typing import List, Optional
8  from fastapi import APIRouter, HTTPException
9  from app.models.schemas import AgentInvokeRequest, AgentType
10 from app.agents.orchestrator import get_orchestrator
11
12 logger = logging.getLogger(__name__)
13 router = APIRouter()
14
15
16 @router.post("/agent/invoke")
17 async def invoke_agent(request: AgentInvokeRequest):
18     """直接调用 Agent 编排器"""
19     try:
20         orchestrator = get_orchestrator()
21         result = await orchestrator.run(
22             query=request.query,
23             session_id=request.session_id or str(uuid.uuid4()),
24             agent_types=request.agent_types,
25             parallel=request.parallel,
26         )
27         return {
28             "run_id": result["run_id"], "session_id": result["session_id"],

```

```

29         "intent": result["intent"], "final_response":
result["final_response"],
30         "execution_summary": result["execution_summary"],
31     }
32     except Exception as e:
33         logger.error(f"[Agent API] failed: {str(e)}")
34         raise HTTPException(status_code=500, detail=str(e))
35
36
37 @router.get("/agent/memory/{session_id}")
38 async def get_session_memory(session_id: str):
39     orchestrator = get_orchestrator()
40     memory = orchestrator.get_session_memory(session_id)
41     return {"session_id": session_id, "conversation_turns": len(memory),
"memory": memory}
42
43
44 @router.delete("/agent/memory/{session_id}")
45 async def clear_session_memory(session_id: str):
46     orchestrator = get_orchestrator()
47     orchestrator.clear_session(session_id)
48     return {"message": "session cleared", "session_id": session_id}
49
50
51 @router.get("/agent/types")
52 async def list_agent_types():
53     return {
54         "agents": [
55             {"type": "researcher", "name": "投研分析Agent",
56             "description": "股票基本面分析", "capabilities": ["stock_analysis",
"financial_report"]},
57             {"type": "risk", "name": "风控预警Agent",
58             "description": "风险识别评估", "capabilities": ["risk_assessment",
"risk_warning"]},
59             {"type": "strategist", "name": "策略生成Agent",
60             "description": "投资策略建议", "capabilities":
["strategy_generation", "portfolio"]},
61         ]
62     }

```

6.19 RAG 知识库 API

代码块

```

1  """
2  backend/app/api/rag.py

```

```

3  RAG 知识库 API
4  """
5  import logging
6  from fastapi import APIRouter, HTTPException
7  from app.models.schemas import RAGQueryRequest, RAGQueryResponse,
KnowledgeBaseUpload
8  from app.rag.retriever import get_hybrid_retriever
9  from app.rag.knowledge_base import get_kb_manager
10
11  logger = logging.getLogger(__name__)
12  router = APIRouter()
13
14
15  @router.post("/rag/query", response_model=RAGQueryResponse)
16  async def rag_query(request: RAGQueryRequest):
17      """RAG 检索接口"""
18      try:
19          retriever = get_hybrid_retriever()
20          chunks = await retriever.retrieve(
21              query=request.query, top_k=request.top_k,
22              score_threshold=request.score_threshold,
23              filters=request.filters, enable_rerank=request.enable_rerank,
24          )
25          return RAGQueryResponse(
26              query=request.query,
27              chunks=[{"content": c["content"], "score": round(c["score"], 3),
28                  "metadata": c.get("metadata", {})} for c in chunks],
29              total_retrieved=len(chunks), rerank_applied=request.enable_rerank,
30          )
31      except Exception as e:
32          logger.error(f"[RAG API] query failed: {str(e)}")
33          raise HTTPException(status_code=500, detail=str(e))
34
35
36  @router.post("/rag/knowledge/add")
37  async def add_knowledge(request: KnowledgeBaseUpload):
38      """添加知识库内容"""
39      try:
40          kb = get_kb_manager()
41          count = await kb.add_text_chunks(
42              texts=[request.content], category=request.category,
43              metadata_list=[{"title": request.title, **(request.metadata or
44                  {})}],
45          )
46          return {"message": f"added {count} chunks", "title": request.title,
47              "category": request.category}
48      except Exception as e:

```

```

47         logger.error(f"[RAG API] add failed: {str(e)}")
48         raise HTTPException(status_code=500, detail=str(e))
49
50
51 @router.get("/rag/knowledge/stats")
52 async def get_knowledge_stats():
53     kb = get_kb_manager()
54     count = kb.count_documents()
55     return {"total_chunks": count, "vectorstore_type": "chroma", "status":
        "healthy"}

```

6.20 LLMOps 监控 API

代码块

```

1  """
2  backend/app/api/llmops.py
3  LLMOps 监控 API
4  """
5  import logging
6  from fastapi import APIRouter, Query
7  from app.llmops.monitor import get_llmops_monitor
8
9  logger = logging.getLogger(__name__)
10 router = APIRouter()
11
12
13 @router.get("/llmops/metrics")
14 async def get_metrics(days: int = Query(7, ge=1, le=90)):
15     """获取 LLMOps 监控指标"""
16     monitor = get_llmops_monitor()
17     return monitor.get_metrics(days=days)
18
19
20 @router.get("/llmops/daily")
21 async def get_daily_stats(days: int = Query(30, ge=1, le=90)):
22     """获取每日统计"""
23     monitor = get_llmops_monitor()
24     return {"daily_stats": monitor.get_daily_stats(days=days)}
25
26
27 @router.get("/llmops/recent")
28 async def get_recent_calls(limit: int = Query(50, ge=1, le=200)):
29     """获取最近调用记录"""
30     monitor = get_llmops_monitor()

```

```
31         return {"recent_calls": monitor.get_recent_calls(limit=limit)}
```

6.21 金融数据查询 API

代码块

```
1  """
2  backend/app/api/data.py
3  金融数据查询 API
4  """
5  import logging
6  from fastapi import APIRouter, HTTPException
7  from app.dataops.pipeline import DataPipeline, PipelineStage,
   wind_stock_quote_extractor
8
9  logger = logging.getLogger(__name__)
10 router = APIRouter()
11
12
13 @router.get("/data/stock/{stock_code}")
14 async def get_stock_info(stock_code: str):
15     """查询股票基本信息"""
16     return {
17         "code": stock_code, "name": f"股票{stock_code}",
18         "industry": "未知行业", "market": "主板",
19         "listing_date": "2010-01-01",
20     }
21
22
23 @router.get("/data/stock/{stock_code}/quote")
24 async def get_stock_quote(stock_code: str):
25     """查询股票实时行情"""
26     return {
27         "code": stock_code, "name": f"股票{stock_code}",
28         "price": 12.50, "change_pct": 1.23, "volume": 50000000,
29         "high": 12.80, "low": 12.30, "open": 12.35,
30         "timestamp": "2024-01-01 09:30:00",
31     }
32
33
34 @router.post("/data/pipeline/run")
35 async def run_data_pipeline(pipeline_name: str = "stock_quote"):
36     """手动触发数据管道"""
37     pipeline = DataPipeline(name=pipeline_name)
38     pipeline.add_stage(PipelineStage(
39         name="extract", description="extract stock data",
```

```
40         extractor=wind_stock_quote_extractor,
41     ))
42     return await pipeline.run()
```

6.22 Redis 连接管理

代码块

```
1  """
2  backend/app/core/redis_client.py
3  Redis 连接管理
4  """
5  import redis.asyncio as redis
6  from typing import Optional
7  from app.config import get_settings
8
9  settings = get_settings()
10 _redis: Optional[redis.Redis] = None
11
12
13 async def get_redis() -> redis.Redis:
14     global _redis
15     if _redis is None:
16         _redis = redis.Redis(
17             host=settings.REDIS_HOST, port=settings.REDIS_PORT,
18             db=settings.REDIS_DB, password=settings.REDIS_PASSWORD,
19             decode_responses=True,
20             max_connections=settings.REDIS_MAX_CONNECTIONS,
21         )
22     return _redis
23
24 async def close_redis():
25     global _redis
26     if _redis:
27         await _redis.close()
28     _redis = None
```

6.23 向量数据库统一接口

代码块

```
1  """
2  backend/app/core/vectorstore.py
3  向量数据库统一接口
```

```

4  """
5  from typing import List, Dict, Any, Optional
6  from app.config import get_settings
7
8  settings = get_settings()
9
10
11  class VectorStoreInterface:
12      """向量数据库统一接口"""
13
14      def __init__(self, store_type: str = None):
15          self.store_type = store_type or settings.VECTORSTORE_TYPE
16          self._client = None
17
18      async def add(
19          self, texts: List[str], embeddings: List[List[float]],
20          metadatas: List[Dict[str, Any]], ids: List[str],
21      ) -> bool:
22          if self.store_type == "chroma":
23              return await self._add_chroma(texts, embeddings, metadatas, ids)
24          elif self.store_type == "milvus":
25              return await self._add_milvus(texts, embeddings, metadatas, ids)
26          return False
27
28      async def search(
29          self, query_embedding: List[float],
30          top_k: int = 10, filters: Optional[Dict] = None,
31      ) -> List[Dict[str, Any]]:
32          if self.store_type == "chroma":
33              return await self._search_chroma(query_embedding, top_k, filters)
34          elif self.store_type == "milvus":
35              return await self._search_milvus(query_embedding, top_k, filters)
36          return []
37
38      async def _add_chroma(self, texts, embeddings, metadatas, ids) -> bool:
39          return True
40
41      async def _search_chroma(self, query_embedding, top_k, filters) ->
42  List[Dict]:
43          return []
44
45      async def _add_milvus(self, texts, embeddings, metadatas, ids) -> bool:
46          return True
47
48      async def _search_milvus(self, query_embedding, top_k, filters) ->
49  List[Dict]:

```



7. 前端完整代码

7.1 前端依赖清单

代码块

```
1  {
2    "name": "financial-ai-agent-frontend",
3    "version": "1.0.0",
4    "type": "module",
5    "scripts": {
6      "dev": "vite",
7      "build": "vue-tsc && vite build",
8      "preview": "vite preview"
9    },
10   "dependencies": {
11     "vue": "^3.4.0",
12     "vue-router": "^4.3.0",
13     "pinia": "^2.1.0",
14     "axios": "^1.7.0",
15     "echarts": "^5.5.0",
16     "@vueuse/core": "^10.11.0",
17     "dayjs": "^1.11.0",
18     "marked": "^12.0.0",
19     "highlight.js": "^11.9.0"
20   },
21   "devDependencies": {
22     "@vitejs/plugin-vue": "^5.0.0",
23     "vite": "^5.3.0",
24     "typescript": "^5.4.0",
25     "vue-tsc": "^2.0.0",
26     "@types/node": "^20.0.0"
27   }
28 }
```

7.2 Vite 构建配置

代码块

```
1  import { defineConfig } from 'vite'
2  import vue from '@vitejs/plugin-vue'
```

```

3  import { resolve } from 'path'
4
5  export default defineConfig({
6    plugins: [vue()],
7    resolve: { alias: { '@': resolve(__dirname, 'src') } },
8    server: {
9      port: 5173,
10     proxy: {
11       '/api': { target: 'http://localhost:8000', changeOrigin: true },
12     },
13   },
14   build: { outDir: 'dist', sourcemap: false },
15 })

```

7.3 TypeScript 配置

代码块

```

1  {
2    "compilerOptions": {
3      "target": "ES2020",
4      "useDefineForClassFields": true,
5      "module": "ESNext",
6      "lib": ["ES2020", "DOM", "DOM.Iterable"],
7      "skipLibCheck": true,
8      "moduleResolution": "bundler",
9      "allowImportingTsExtensions": true,
10     "resolveJsonModule": true,
11     "isolatedModules": true,
12     "noEmit": true,
13     "jsx": "preserve",
14     "strict": true,
15     "paths": { "@/*": [". /src/*"] }
16   },
17   "include": ["src/**/*.ts", "src/**/*.d.ts", "src/**/*.tsx", "src/**/*.vue"],
18   "references": [{ "path": "./tsconfig.node.json" }]
19 }

```

7.4 Vue3 应用入口

代码块

```

1  import { createApp } from 'vue'
2  import { createPinia } from 'pinia'
3  import App from './App.vue'

```

```
4 import router from './router'
5 import './style.css'
6
7 const app = createApp(App)
8 app.use(createPinia())
9 app.use(router)
10 app.mount('#app')
```

7.5 路由配置

代码块

```
1 import { createRouter, createWebHistory } from 'vue-router'
2 import ChatView from './views/ChatView.vue'
3 import DashboardView from './views/DashboardView.vue'
4 import ReportView from './views/ReportView.vue'
5 import KnowledgeView from './views/KnowledgeView.vue'
6 import LLMOpsView from './views/LLMOpsView.vue'
7
8 const router = createRouter({
9   history: createWebHistory(),
10  routes: [
11    { path: '/', name: 'chat', component: ChatView },
12    { path: '/dashboard', name: 'dashboard', component: DashboardView },
13    { path: '/report', name: 'report', component: ReportView },
14    { path: '/knowledge', name: 'knowledge', component: KnowledgeView },
15    { path: '/llmops', name: 'llmops', component: LLMOpsView },
16  ],
17 })
18
19 export default router
```

7.6 全局样式

代码块

```
1 *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
2 html, body, #app { height: 100%; overflow: hidden; }
3 body {
4   font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto,
5     'PingFang SC', 'Microsoft YaHei', sans-serif;
6   background: #0a0e1a;
7   color: #e0e6ed;
8   line-height: 1.6;
9   -webkit-font-smoothing: antialiased;
```

```

10 }
11 ::-webkit-scrollbar { width: 6px; height: 6px; }
12 ::-webkit-scrollbar-track { background: rgba(255,255,255,0.03); }
13 ::-webkit-scrollbar-thumb { background: rgba(99,179,237,0.3); border-radius:
  3px; }
14 ::-webkit-scrollbar-thumb:hover { background: rgba(99,179,237,0.5); }
15 code, pre { font-family: 'Fira Code', 'JetBrains Mono', Consolas, monospace; }

```

7.7 Axios 统一封装

代码块

```

1  import axios from 'axios'
2
3  const API_BASE = import.meta.env.VITE_API_BASE ||
  'http://localhost:8000/api/v1'
4
5  const apiClient = axios.create({
6    baseURL: API_BASE,
7    timeout: 120000,
8    headers: { 'Content-Type': 'application/json' },
9  })
10
11  apiClient.interceptors.request.use(config => config, error =>
  Promise.reject(error))
12  apiClient.interceptors.response.use(
13    response => response.data,
14    error => {
15      const message = error.response?.data?.detail || error.message
16      return Promise.reject(new Error(message))
17    }
18  )
19
20  export default apiClient

```

7.8 对话 API 封装

代码块

```

1  import apiClient from './index'
2
3  export interface ChatMessage {
4    role: 'user' | 'assistant'
5    content: string
6    agent_type?: string

```

```

7   sources?: Array<{ content: string; score: number }>
8   token_usage?: { total: number }
9   latency_ms?: number
10  timestamp?: string
11  }
12
13  export interface ChatRequest {
14    session_id: string
15    message: string
16    agent_type?: string
17    stream: boolean
18    enable_rag: boolean
19  }
20
21  export interface ChatResponse {
22    session_id: string
23    message: string
24    agent_type: string
25    sources?: Array<{ content: string; score: number }>
26    token_usage?: { total: number }
27    latency_ms?: number
28  }
29
30  export const chatApi = {
31    send: (data: ChatRequest) => apiClient.post<ChatResponse>('/chat', data),
32  }
33
34  export const API_BASE = import.meta.env.VITE_API_BASE ||
    'http://localhost:8000/api/v1'

```

7.9 LLMOps API 类型定义

代码块

```

1  import apiClient from './index'
2
3  export interface LLMOpsMetrics {
4    all_time: { total_calls: number; total_tokens: number; total_cost_usd:
      number; avg_latency_ms: number; success_rate: number }
5    today: { calls: number; tokens: number; cost_usd: number; avg_latency_ms:
      number; success_rate: number }
6  }
7
8  export interface DailyStats {
9    date: string; calls: number; tokens: number; cost_usd: number;
    avg_latency_ms: number; success_rate: number

```

```

10 }
11
12 export interface RecentCall {
13   call_id: string; model: string; tokens: number; latency_ms: number;
14   cost_usd: number
15   success: boolean; error?: string; timestamp: string
16 }
17
18 export const llmopsApi = {
19   getMetrics: (days = 7) => apiClient.get<LLMOpsMetrics>('/llmops/metrics', {
20     params: { days } }),
21   getDailyStats: (days = 30) => apiClient.get<{ daily_stats: DailyStats[] }>
22     ('/llmops/daily', { params: { days } }),
23   getRecentCalls: (limit = 50) => apiClient.get<{ recent_calls: RecentCall[] }>
24     ('/llmops/recent', { params: { limit } }),
25 }

```

7.10 Agent API 封装

代码块

```

1 import apiClient from './index'
2
3 export const agentApi = {
4   invoke: (data: { session_id: string; query: string; agent_types?: string[];
5     parallel?: boolean }) =>
6     apiClient.post('/agent/invoke', data),
7   getMemory: (sessionId: string) =>
8     apiClient.get(`/agent/memory/${sessionId}`),
9   clearMemory: (sessionId: string) =>
10     apiClient.delete(`/agent/memory/${sessionId}`),
11   listTypes: () => apiClient.get('/agent/types'),
12 }

```

7.11 对话状态管理

代码块

```

1 import { defineStore } from 'pinia'
2 import { ref } from 'vue'
3
4 export interface ChatMessage {
5   role: 'user' | 'assistant'
6   content: string
7   agent_type?: string

```

```

8     sources?: any[]
9     token_usage?: { total: number }
10    latency_ms?: number
11  }
12
13  export const useChatStore = defineStore('chat', () => {
14    const messages = ref<ChatMessage[]>([])
15    const sessionId = ref(`session_${Date.now()}`)
16    const isLoading = ref(false)
17    const enableRAG = ref(true)
18
19    function addMessage(msg: ChatMessage) { messages.value.push(msg) }
20    function clearMessages() { messages.value = []; sessionId.value =
    `session_${Date.now()}` }
21
22    return { messages, sessionId, isLoading, enableRAG, addMessage,
    clearMessages }
23  })

```

7.12 LLMOps 状态管理

代码块

```

1  import { defineStore } from 'pinia'
2  import { ref } from 'vue'
3
4  export const useLLMOpsStore = defineStore('llmops', () => {
5    const metrics = ref<any>(null)
6    const dailyStats = ref<any[]>([])
7    const recentCalls = ref<any[]>([])
8
9    function setMetrics(data: any) { metrics.value = data }
10   function setDailyStats(data: any[]) { dailyStats.value = data }
11   function setRecentCalls(data: any[]) { recentCalls.value = data }
12
13   return { metrics, dailyStats, recentCalls, setMetrics, setDailyStats,
    setRecentCalls }
14 })

```

7.13 根组件与侧边导航

代码块

```

1  <template>
2    <div id="app" class="app-container">

```

```

3      <aside class="sidebar">
4          <div class="sidebar-header">
5              <h1 class="logo"><img alt="AI Agent icon" data-bbox="375 75 395 90"/> 金融AI Agent</h1>
6              <p class="subtitle">LLM + Agent + RAG</p>
7          </div>
8          <nav class="nav-menu">
9              <router-link to="/" class="nav-item" active-class="active">
10                 <span class="nav-icon"><img alt="Chat icon" data-bbox="455 175 475 190"/></span>智能对话
11             </router-link>
12             <router-link to="/dashboard" class="nav-item" active-class="active">
13                 <span class="nav-icon"><img alt="Dashboard icon" data-bbox="455 235 475 250"/></span>数据看板
14             </router-link>
15             <router-link to="/report" class="nav-item" active-class="active">
16                 <span class="nav-icon"><img alt="Report icon" data-bbox="455 290 475 305"/></span>报告中心
17             </router-link>
18             <router-link to="/knowledge" class="nav-item" active-class="active">
19                 <span class="nav-icon"><img alt="Knowledge icon" data-bbox="455 345 475 360"/></span>知识库
20             </router-link>
21             <router-link to="/llmops" class="nav-item" active-class="active">
22                 <span class="nav-icon"><img alt="LLMops icon" data-bbox="455 400 475 415"/></span>LLMops监控
23             </router-link>
24         </nav>
25         <div class="sidebar-footer">
26             <div class="status-indicator">
27                 <span class="status-dot"></span>系统正常
28             </div>
29         </div>
30     </aside>
31     <main class="main-content">
32         <router-view />
33     </main>
34 </div>
35 </template>
36
37 <style scoped>
38 .app-container { display: flex; height: 100vh; background: #0a0e1a; color:
39 #e0e6ed; }
40 .sidebar { width: 240px; background: linear-gradient(180deg, #0f1629 0%,
41 #1a1f35 100%);
42     border-right: 1px solid rgba(99,179,237,0.15); display: flex; flex-
43     direction: column; padding: 20px 0; }
44 .sidebar-header { padding: 0 20px 20px; border-bottom: 1px solid
45     rgba(99,179,237,0.1); }
46 .logo { font-size: 18px; font-weight: 700; color: #63b3ed; margin: 0; }
47 .subtitle { font-size: 11px; color: #718096; margin: 4px 0 0; }
48 .nav-menu { flex: 1; padding: 16px 0; }
49 .nav-item { display: flex; align-items: center; gap: 10px; padding: 12px 20px;

```

```

46     color: #a0aec0; text-decoration: none; font-size: 14px; transition: all 0.2s;
47     border-left: 3px solid transparent; }
48 .nav-item:hover { background: rgba(99,179,237,0.1); color: #63b3ed; }
49 .nav-item.active { background: rgba(99,179,237,0.15); color: #63b3ed; border-
left-color: #63b3ed; }
50 .sidebar-footer { padding: 16px 20px; border-top: 1px solid
rgba(99,179,237,0.1); }
51 .status-indicator { display: flex; align-items: center; gap: 8px; font-size:
12px; color: #68d391; }
52 .status-dot { width: 8px; height: 8px; border-radius: 50%; background:
#68d391; box-shadow: 0 0 8px #68d391; }
53 .main-content { flex: 1; overflow: hidden; display: flex; flex-direction:
column; }
54 </style>

```

7.14 智能对话页面

代码块

```

1  <template>
2    <div class="chat-view">
3      <header class="chat-header">
4        <div class="header-left">
5          <h2>💬 智能投研对话</h2>
6          <span class="model-tag">Qwen-Plus + RAG</span>
7        </div>
8        <div class="header-right">
9          <button class="btn-icon" @click="clearChat" title="清空对话">🗑️</button>
10       </div>
11     </header>
12
13     <div class="chat-body" ref="chatBodyRef">
14       <div v-if="messages.length === 0" class="welcome-panel">
15         <div class="welcome-icon">🤖</div>
16         <h3>金融分析 AI Agent</h3>
17         <p>基于 LLM + Agent + RAG 的智能投研助手</p>
18         <div class="quick-prompts">
19           <div class="prompt-label">试试这样问我:</div>
20           <button class="prompt-chip" @click="quickAsk('分析贵州茅台600519的基本面
和估值')">
21             📈 分析贵州茅台的基本面
22           </button>
23           <button class="prompt-chip" @click="quickAsk('对宁德时代进行风险评估')">
24             ⚠️ 评估宁德时代的风险
25           </button>

```

```

26     <button class="prompt-chip" @click="quickAsk('分析当前市场行情，给出投资
建议')">
27         💡 分析市场并给出投资建议
28     </button>
29 </div>
30 </div>
31
32 <div v-else class="message-list">
33     <div v-for="(msg, idx) in messages" :key="idx" class="message-item"
: class="msg.role">
34         <div class="message-avatar">{{ msg.role === 'user' ? '👤' : '🤖' }}
</div>
35         <div class="message-content">
36             <div class="message-header">
37                 <span class="sender-name">{{ msg.role === 'user' ? '你' : 'AI
Agent' }}</span>
38                 <span v-if="msg.agent_type" class="agent-badge">{{
msg.agent_type }}</span>
39             </div>
40             <div class="message-text" v-html="renderMarkdown(msg.content)">
</div>
41             <div v-if="msg.sources && msg.sources.length > 0" class="sources-
panel">
42                 <div class="sources-title">📖 参考来源</div>
43                 <div v-for="(src, sidx) in msg.sources" :key="sidx"
class="source-item">
44                     <span class="source-score">{{ (src.score * 100).toFixed(0) }}%
</span>
45                     <span class="source-content">{{ src.content }}</span>
46                 </div>
47             </div>
48             <div v-if="msg.token_usage || msg.latency_ms" class="message-meta">
49                 <span v-if="msg.token_usage">💎 {{ msg.token_usage.total }}
tokens</span>
50                 <span v-if="msg.latency_ms">🕒 {{ msg.latency_ms }}ms</span>
51             </div>
52         </div>
53     </div>
54
55     <div v-if="isLoading" class="message-item assistant">
56         <div class="message-avatar">🤖</div>
57         <div class="message-content">
58             <div class="typing-indicator"><span></span><span></span><span></span><span>
</span></div>
59         </div>
60     </div>
61 </div>

```

```

62     </div>
63
64     <div class="chat-footer">
65         <div class="input-toolbar">
66             <label class="toolbar-item">
67                 <input type="checkbox" v-model="enableRAG" />
68                 <span>启用RAG增强</span>
69             </label>
70         </div>
71         <div class="input-row">
72             <textarea v-model="inputMessage" class="input-area" placeholder="输入你
的金融分析问题..."
73                 rows="1" @keydown.enter.exact.prevent="sendMessage"
@input="autoResize" ref="inputRef"></textarea>
74             <button class="send-btn" @click="sendMessage"
:disabled="!inputMessage.trim() || isLoading">
75                 {{ isLoading ? '⌚' : '🚀' }}
76             </button>
77         </div>
78     </div>
79 </div>
80 </template>
81
82 <script setup lang="ts">
83 import { ref, nextTick } from 'vue'
84 import { marked } from 'marked'
85 import { chatApi, ChatMessage } from '@/api/chat'
86
87 const messages = ref<Array<ChatMessage & { sources?: any[]; token_usage?: any;
latency_ms?: number }>>([])
88 const inputMessage = ref('')
89 const isLoading = ref(false)
90 const enableRAG = ref(true)
91 const sessionId = ref(`session_${Date.now()}`)
92 const chatBodyRef = ref<HTMLElement>()
93 const inputRef = ref<HTMLTextAreaElement>()
94
95 function renderMarkdown(text: string) {
96     return marked.parse(text, { async: false }) as string
97 }
98
99 function autoResize() {
100     const el = inputRef.value
101     if (el) { el.style.height = 'auto'; el.style.height =
Math.min(el.scrollHeight, 200) + 'px' }
102 }
103

```

```
104 async function sendMessage() {
105     const text = inputMessage.value.trim()
106     if (!text || isLoading.value) return
107
108     messages.value.push({ role: 'user', content: text })
109     inputMessage.value = ''
110     isLoading.value = true
111     await nextTick()
112     scrollToBottom()
113
114     try {
115         const res = await chatApi.send({
116             session_id: sessionId.value,
117             message: text,
118             enable_rag: enableRAG.value,
119             stream: false,
120         })
121         messages.value.push({
122             role: 'assistant', content: res.message,
123             agent_type: res.agent_type, sources: res.sources,
124             token_usage: res.token_usage, latency_ms: res.latency_ms,
125         })
126     } catch (e: any) {
127         messages.value.push({ role: 'assistant', content: `❌ 出错了: ${e.message}`
128     })
129     } finally {
130         isLoading.value = false
131         await nextTick()
132         scrollToBottom()
133     }
134
135     function quickAsk(question: string) { inputMessage.value = question;
136         sendMessage() }
137     function clearChat() { messages.value = []; sessionId.value =
138         `session_${Date.now()}` }
139     function scrollToBottom() {
140         if (chatBodyRef.value) chatBodyRef.value.scrollTop =
141         chatBodyRef.value.scrollHeight
142     }
143 }
144 </script>
145
146 <style scoped>
147 .chat-view { display: flex; flex-direction: column; height: 100%; background:
148 #0a0e1a; }
149 .chat-header { display: flex; align-items: center; justify-content: space-
150 between;
```

```
145     padding: 16px 24px; border-bottom: 1px solid rgba(99,179,237,0.15);
146     background: linear-gradient(90deg, rgba(99,179,237,0.08) 0%, transparent
147     100%); }
148 .header-left { display: flex; align-items: center; gap: 12px; }
149 .header-left h2 { font-size: 18px; font-weight: 600; color: #e0e6ed; margin:
150 0; }
151 .model-tag { padding: 4px 10px; background: rgba(99,179,237,0.2); border-
152 radius: 20px; font-size: 12px; color: #63b3ed; }
153 .btn-icon { background: none; border: none; cursor: pointer; font-size: 18px;
154 padding: 8px; border-radius: 8px; }
155 .btn-icon:hover { background: rgba(99,179,237,0.15); }
156 .chat-body { flex: 1; overflow-y: auto; padding: 24px; }
157 .welcome-panel { display: flex; flex-direction: column; align-items: center;
158 justify-content: center; height: 100%; text-align: center; color: #a0aec0; }
159 .welcome-icon { font-size: 64px; margin-bottom: 16px; }
160 .welcome-panel h3 { font-size: 24px; color: #e0e6ed; margin: 0 0 8px; }
161 .welcome-panel p { font-size: 14px; color: #718096; margin: 0 0 32px; }
162 .quick-prompts { display: flex; flex-direction: column; gap: 12px; align-
163 items: center; }
164 .prompt-label { font-size: 13px; color: #718096; }
165 .prompt-chip { padding: 10px 20px; background: rgba(99,179,237,0.1); border:
166 1px solid rgba(99,179,237,0.25);
167 border-radius: 24px; color: #a0aec0; cursor: pointer; font-size: 13px;
168 transition: all 0.2s; max-width: 400px; }
169 .prompt-chip:hover { background: rgba(99,179,237,0.2); border-color:
170 rgba(99,179,237,0.5); color: #63b3ed; transform: translateY(-2px); }
171 .message-list { display: flex; flex-direction: column; gap: 20px; }
172 .message-item { display: flex; gap: 12px; max-width: 80%; }
173 .message-item.user { align-self: flex-end; flex-direction: row-reverse; }
174 .message-avatar { width: 36px; height: 36px; border-radius: 50%; display:
175 flex; align-items: center; justify-content: center; font-size: 18px; flex-
176 shrink: 0; background: rgba(99,179,237,0.15); }
177 .message-item.assistant .message-avatar { background: rgba(104,211,145,0.15); }
178 .message-content { flex: 1; background: rgba(255,255,255,0.04); border: 1px
179 solid rgba(255,255,255,0.08); border-radius: 12px; padding: 14px 18px; }
180 .message-item.user .message-content { background: rgba(99,179,237,0.12);
181 border-color: rgba(99,179,237,0.25); }
182 .message-header { display: flex; align-items: center; gap: 8px; margin-bottom:
183 8px; }
184 .sender-name { font-size: 13px; font-weight: 600; color: #a0aec0; }
185 .agent-badge { padding: 2px 8px; background: rgba(104,211,145,0.2); border-
186 radius: 10px; font-size: 11px; color: #68d391; }
187 .message-text { font-size: 14px; line-height: 1.7; color: #e0e6ed; word-break:
188 break-word; }
189 .message-text :deep(strong) { color: #63b3ed; }
190 .message-text :deep(pre) { background: rgba(0,0,0,0.3); padding: 12px; border-
191 radius: 8px; overflow-x: auto; }
```

```
175 .message-text :deep(table) { width: 100%; border-collapse: collapse; margin:
176 8px 0; }
177 .message-text :deep(th), .message-text :deep(td) { padding: 8px 12px; border:
178 1px solid rgba(255,255,255,0.1); font-size: 13px; }
179 .message-text :deep(th) { background: rgba(99,179,237,0.1); color: #63b3ed; }
180 .sources-panel { margin-top: 12px; padding: 10px; background:
181 rgba(99,179,237,0.08); border-radius: 8px; border: 1px solid
182 rgba(99,179,237,0.15); }
183 .sources-title { font-size: 12px; color: #63b3ed; margin-bottom: 8px; font-
184 weight: 600; }
185 .source-item { display: flex; align-items: flex-start; gap: 8px; padding: 4px
186 0; font-size: 12px; color: #718096; }
187 .source-score { color: #68d391; font-weight: 600; flex-shrink: 0; }
188 .message-meta { display: flex; gap: 12px; margin-top: 8px; font-size: 12px;
189 color: #718096; }
190 .typing-indicator { display: flex; gap: 4px; padding: 4px 0; }
191 .typing-indicator span { width: 8px; height: 8px; border-radius: 50%;
192 background: #63b3ed; animation: bounce 1.4s infinite ease-in-out; }
193 .typing-indicator span:nth-child(1) { animation-delay: -0.32s; }
194 .typing-indicator span:nth-child(2) { animation-delay: -0.16s; }
195 @keyframes bounce { 0%, 80%, 100% { transform: scale(0.6); opacity: 0.4; } 40%
196 { transform: scale(1); opacity: 1; } }
197 .chat-footer { border-top: 1px solid rgba(99,179,237,0.15); padding: 16px
198 24px; background: rgba(15,22,41,0.8); }
199 .input-toolbar { display: flex; gap: 16px; margin-bottom: 10px; }
200 .toolbar-item { display: flex; align-items: center; gap: 6px; font-size: 12px;
201 color: #718096; cursor: pointer; }
202 .toolbar-item input[type="checkbox"] { accent-color: #63b3ed; }
203 .input-row { display: flex; gap: 12px; align-items: flex-end; }
204 .input-area { flex: 1; background: rgba(255,255,255,0.06); border: 1px solid
205 rgba(99,179,237,0.2);
206 border-radius: 12px; padding: 12px 16px; color: #e0e6ed; font-size: 14px;
207 resize: none; outline: none;
208 font-family: inherit; line-height: 1.5; transition: border-color 0.2s; }
209 .input-area:focus { border-color: rgba(99,179,237,0.5); background:
210 rgba(99,179,237,0.06); }
211 .input-area::placeholder { color: #4a5568; }
212 .send-btn { width: 44px; height: 44px; border-radius: 12px; border: none;
213 background: linear-gradient(135deg, #3182ce, #63b3ed); color: white; font-
214 size: 18px;
215 cursor: pointer; transition: all 0.2s; display: flex; align-items: center;
216 justify-content: center; flex-shrink: 0; }
217 .send-btn:hover:not(:disabled) { transform: translateY(-2px); box-shadow: 0
218 4px 12px rgba(99,179,237,0.4); }
219 .send-btn:disabled { opacity: 0.4; cursor: not-allowed; }
220 </style>
```

7.15 LLMOps 监控面板

代码块

```
1  <template>
2    <div class="llmops-view">
3      <header class="page-header">
4        <h2>⚙️ LLMOps 监控面板</h2>
5        <button class="btn-refresh" @click="loadData">🔄 刷新</button>
6      </header>
7
8      <div class="metrics-grid">
9        <div class="metric-card">
10          <div class="metric-label">今日调用次数</div>
11          <div class="metric-value">{{ metrics?.today?.calls || 0 }}</div>
12          <div class="metric-sub">成功率 {{ metrics?.today?.success_rate || 0 }}%
13        </div>
14        <div class="metric-card">
15          <div class="metric-label">今日 Token 消耗</div>
16          <div class="metric-value">{{ formatNumber(metrics?.today?.tokens || 0)
17        </div>
18        <div class="metric-card">
19          <div class="metric-label">今日成本</div>
20          <div class="metric-value">${{ (metrics?.today?.cost_usd ||
21        0).toFixed(4) }}</div>
22          <div class="metric-sub">累计 ${{ (metrics?.all_time?.total_cost_usd ||
23        0).toFixed(4) }}</div>
24        </div>
25        <div class="metric-card">
26          <div class="metric-label">累计调用</div>
27          <div class="metric-value">{{
28        formatNumber(metrics?.all_time?.total_calls || 0) }}</div>
29          <div class="metric-sub">累计 Token {{
30        formatNumber(metrics?.all_time?.total_tokens || 0) }}</div>
31        </div>
32      </div>
33
34      <div class="section">
35        <h3 class="section-title">📄 最近调用记录</h3>
36        <div class="table-wrapper">
37          <table class="data-table">
38            <thead><tr><th>时间</th><th>模型</th><th>Token数</th><th>延迟</th><th>
39            成本</th><th>状态</th></tr></thead>
```

```

36     <tbody>
37       <tr v-for="call in recentCalls" :key="call.call_id">
38         <td>{{ call.timestamp }}</td>
39         <td><code>{{ call.model }}</code></td>
40         <td>{{ formatNumber(call.tokens) }}</td>
41         <td>{{ call.latency_ms }}ms</td>
42         <td>${{ call.cost_usd.toFixed(6) }}</td>
43         <td><span :class="['status-tag', call.success ? 'success' :
'error']">{{ call.success ? '✅ 成功' : '❌ 失败' }}</span></td>
44       </tr>
45     </tbody>
46   </table>
47 </div>
48 </div>
49
50 <div class="section">
51   <h3 class="section-title">📈 每日调用趋势（前14天）</h3>
52   <div class="bar-chart">
53     <div v-for="day in dailyStats.slice(-14)" :key="day.date" class="bar-
item">
54       <div class="bar" :style="{ height: (day.calls / maxDailyCalls * 100)
+ '%' }" :title="day.date + ': ' + day.calls + '次'">
55         <div class="bar-label">{{ day.date.slice(5) }}</div>
56       </div>
57     </div>
58   </div>
59 </div>
60 </template>
61
62 <script setup lang="ts">
63 import { ref, onMounted, computed } from 'vue'
64 import { llmopsApi, type LLMOpsMetrics, type DailyStats, type RecentCall }
from '@/api/llmops'
65
66 const metrics = ref<LLMOpsMetrics>()
67 const recentCalls = ref<RecentCall[]>([])
68 const dailyStats = ref<DailyStats[]>([])
69
70 const maxDailyCalls = computed(() => {
71   if (!dailyStats.value.length) return 1
72   return Math.max(...dailyStats.value.map(d => d.calls), 1)
73 })
74
75 function formatNumber(n: number) {
76   if (n >= 1000000) return (n / 1000000).toFixed(1) + 'M'
77   if (n >= 1000) return (n / 1000).toFixed(1) + 'K'
78   return n.toString()

```

```

79 }
80
81 async function loadData() {
82   try {
83     const [m, r, d] = await Promise.all([
84       llmopsApi.getMetrics(7),
85       llmopsApi.getRecentCalls(30),
86       llmopsApi.getDailyStats(30),
87     ])
88     metrics.value = m
89     recentCalls.value = r.recent_calls
90     dailyStats.value = d.daily_stats
91   } catch (e) { console.error('LLMOps load error:', e) }
92 }
93
94 onMounted(loadData)
95 </script>
96
97 <style scoped>
98 .llmops-view { padding: 24px; height: 100%; overflow-y: auto; }
99 .page-header { display: flex; align-items: center; justify-content: space-
between; margin-bottom: 24px; }
100 .page-header h2 { font-size: 20px; font-weight: 600; color: #e0e6ed; margin:
0; }
101 .btn-refresh { padding: 8px 16px; background: rgba(99,179,237,0.15); border:
1px solid rgba(99,179,237,0.3); border-radius: 8px; color: #63b3ed; cursor:
pointer; font-size: 13px; }
102 .metrics-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap:
16px; margin-bottom: 24px; }
103 .metric-card { background: linear-gradient(135deg, rgba(99,179,237,0.1),
rgba(99,179,237,0.05)); border: 1px solid rgba(99,179,237,0.2); border-radius:
12px; padding: 20px; text-align: center; }
104 .metric-label { font-size: 12px; color: #718096; margin-bottom: 8px; }
105 .metric-value { font-size: 28px; font-weight: 700; color: #63b3ed; margin-
bottom: 4px; }
106 .metric-sub { font-size: 11px; color: #4a5568; }
107 .section { background: rgba(255,255,255,0.03); border: 1px solid
rgba(255,255,255,0.08); border-radius: 12px; padding: 20px; margin-bottom:
20px; }
108 .section-title { font-size: 15px; font-weight: 600; color: #e0e6ed; margin: 0
0 16px; }
109 .table-wrapper { overflow-x: auto; }
110 .data-table { width: 100%; border-collapse: collapse; font-size: 13px; }
111 .data-table th { text-align: left; padding: 10px 12px; color: #718096; font-
weight: 500; border-bottom: 1px solid rgba(255,255,255,0.08); }
112 .data-table td { padding: 10px 12px; color: #a0aec0; border-bottom: 1px solid
rgba(255,255,255,0.04); }

```

```

113 .data-table code { font-size: 12px; color: #68d391; background:
    rgba(104,211,145,0.1); padding: 2px 6px; border-radius: 4px; }
114 .status-tag { padding: 2px 8px; border-radius: 10px; font-size: 12px; }
115 .status-tag.success { background: rgba(104,211,145,0.15); color: #68d391; }
116 .status-tag.error { background: rgba(252,129,129,0.15); color: #fc8181; }
117 .bar-chart { height: 160px; display: flex; align-items: flex-end; gap: 6px; }
118 .bar-item { flex: 1; display: flex; flex-direction: column; align-items:
    center; height: 100%; justify-content: flex-end; }
119 .bar { width: 100%; background: linear-gradient(to top, #3182ce, #63b3ed);
    border-radius: 4px 4px 0 0; min-height: 4px; transition: height 0.3s; opacity:
    0.8; }
120 .bar:hover { opacity: 1; }
121 .bar-label { font-size: 10px; color: #4a5568; margin-top: 4px; }
122 </style>

```

7.16 数据看板页面

代码块

```

1 <template>
2   <div class="dashboard-view">
3     <header class="page-header"><h2><img alt="Financial Data Dashboard Icon" data-bbox="488 468 508 483"/> 金融数据看板</h2></header>
4     <div class="stock-cards">
5       <div v-for="stock in mockStocks" :key="stock.code" class="stock-card">
6         <div class="stock-top">
7           <div class="stock-name">{{ stock.name }}</div>
8           <div class="stock-code">{{ stock.code }}</div>
9         </div>
10        <div class="stock-price">¥{{ stock.price.toFixed(2) }}</div>
11        <div class="stock-change" :class="stock.changePct >= 0 ? 'up' :
'down'">
12          {{ stock.changePct >= 0 ? '+' : '' }}{{ stock.changePct.toFixed(2)
}}%
13        </div>
14        <div class="stock-vol">成交量: {{ (stock.volume / 100000000).toFixed(2)
}}亿</div>
15      </div>
16    </div>
17    <div class="dashboard-bottom">
18      <div class="panel">
19        <h3>系统架构说明</h3>
20        <div class="arch-list">
21          <div class="arch-item"><img alt="Agent编排层图标" data-bbox="225 878 245 893"/> <strong>Agent 编排层</strong>: 多 Agent 协
同, 意图分类, 任务调度</div>
22          <div class="arch-item"><img alt="RAG知识增强图标" data-bbox="225 918 245 933"/> <strong>RAG 知识增强</strong>: 混合检索, 向量
召回, GRAG 生成增强</div>

```

```
23     <div class="arch-item">◆ <strong>LLM 大模型层</strong>:
Qwen/DeepSeek, Function Calling, 工具调用</div>
24     <div class="arch-item">◆ <strong>LLMOps 监控</strong>: Token 计量, 成
本追踪, Prometheus 指标</div>
25     <div class="arch-item">◆ <strong>DataOps 数据层</strong>: Prefect 管
道, ETL 编排, 数据质量校验</div>
26     </div>
27     </div>
28     </div>
29     </div>
30 </template>
31
32 <script setup lang="ts">
33 import { ref } from 'vue'
34 const mockStocks = ref([
35   { code: '600519', name: '贵州茅台', price: 1688.00, changePct: 1.25, volume:
28000000000 },
36   { code: '000858', name: '五粮液', price: 142.50, changePct: -0.83, volume:
12000000000 },
37   { code: '300750', name: '宁德时代', price: 198.30, changePct: 2.15, volume:
35000000000 },
38   { code: '601318', name: '中国平安', price: 45.80, changePct: 0.55, volume:
22000000000 },
39 ])
40 </script>
41
42 <style scoped>
43 .dashboard-view { padding: 24px; height: 100%; overflow-y: auto; }
44 .page-header h2 { font-size: 20px; font-weight: 600; color: #e0e6ed; margin: 0
0 24px; }
45 .stock-cards { display: grid; grid-template-columns: repeat(4, 1fr); gap:
16px; margin-bottom: 24px; }
46 .stock-card { background: linear-gradient(135deg, rgba(99,179,237,0.1),
rgba(99,179,237,0.03)); border: 1px solid rgba(99,179,237,0.2); border-radius:
12px; padding: 20px; }
47 .stock-top { display: flex; justify-content: space-between; align-items:
center; margin-bottom: 12px; }
48 .stock-name { font-size: 15px; font-weight: 600; color: #e0e6ed; }
49 .stock-code { font-size: 12px; color: #718096; }
50 .stock-price { font-size: 24px; font-weight: 700; color: #e0e6ed; margin-
bottom: 6px; }
51 .stock-change { font-size: 14px; font-weight: 600; margin-bottom: 6px; }
52 .stock-change.up { color: #fc8181; }
53 .stock-change.down { color: #68d391; }
54 .stock-vol { font-size: 12px; color: #4a5568; }
55 .panel { background: rgba(255,255,255,0.03); border: 1px solid
rgba(255,255,255,0.08); border-radius: 12px; padding: 20px; }
```

```

56 .panel h3 { font-size: 15px; color: #e0e6ed; margin: 0 0 16px; }
57 .arch-list { display: flex; flex-direction: column; gap: 10px; }
58 .arch-item { font-size: 13px; color: #a0aec0; line-height: 1.6; }
59 .arch-item strong { color: #63b3ed; }
60 </style>

```

7.17 报告中心页面

代码块

```

1  <template>
2    <div class="report-view">
3      <header class="page-header">
4        <h2>📝 报告中心</h2>
5        <button class="btn-primary" @click="generateReport">+ 生成报告</button>
6      </header>
7      <div class="report-list">
8        <div v-for="report in reports" :key="report.id" class="report-card">
9          <div class="report-header">
10             <div class="report-title">{{ report.title }}</div>
11             <span class="report-type-badge">{{ report.type }}</span>
12          </div>
13          <div class="report-meta">
14            <span>📅 {{ report.date }}</span>
15            <span>📊 {{ report.stock }}</span>
16          </div>
17          <p class="report-summary">{{ report.summary }}</p>
18        </div>
19      </div>
20    </div>
21  </template>
22
23  <script setup lang="ts">
24    import { ref } from 'vue'
25    const reports = ref([
26      { id: 1, title: '贵州茅台深度投研报告', type: '投研', date: '2024-01-15',
27        stock: '600519', summary: '基本面稳健，估值合理偏高，建议关注。' },
28      { id: 2, title: '宁德时代风险评估', type: '风控', date: '2024-01-14', stock:
29        '300750', summary: '市场竞争加剧，供应链风险可控。' },
30      { id: 3, title: '科技板块投资策略', type: '策略', date: '2024-01-13', stock: '板
31        块', summary: 'TMT板块轮动，建议均衡配置。' },
32    ])
33    function generateReport() { alert('调用报告生成 Agent...') }
34  </script>
35
36  <style scoped>

```

```

34 .report-view { padding: 24px; height: 100%; overflow-y: auto; }
35 .page-header { display: flex; justify-content: space-between; align-items:
center; margin-bottom: 24px; }
36 .page-header h2 { font-size: 20px; color: #e0e6ed; margin: 0; }
37 .btn-primary { padding: 8px 20px; background: #3182ce; border: none; border-
radius: 8px; color: white; cursor: pointer; font-size: 14px; }
38 .report-list { display: flex; flex-direction: column; gap: 16px; }
39 .report-card { background: rgba(255,255,255,0.04); border: 1px solid
rgba(255,255,255,0.08); border-radius: 12px; padding: 20px; }
40 .report-header { display: flex; justify-content: space-between; align-items:
center; margin-bottom: 8px; }
41 .report-title { font-size: 15px; font-weight: 600; color: #e0e6ed; }
42 .report-type-badge { padding: 3px 10px; background: rgba(99,179,237,0.2);
border-radius: 12px; font-size: 12px; color: #63b3ed; }
43 .report-meta { display: flex; gap: 16px; font-size: 12px; color: #718096;
margin-bottom: 10px; }
44 .report-summary { font-size: 13px; color: #a0aec0; margin: 0; line-height:
1.6; }
45 </style>

```

7.18 知识库管理页面

代码块

```

1 <template>
2   <div class="knowledge-view">
3     <header class="page-header">
4       <h2><img alt="book icon" data-bbox="225 585 245 600"/> 知识库管理</h2>
5       <button class="btn-primary" @click="showAddDialog = true">+ 上传文档
</button>
6     </header>
7     <div class="stats-row">
8       <div class="stat-item"><div class="stat-value">{{ totalChunks }}</div>
<div class="stat-label">知识片段数</div></div>
9       <div class="stat-item"><div class="stat-value">ChromaDB</div><div
class="stat-label">向量引擎</div></div>
10      <div class="stat-item"><div class="stat-value">cosine</div><div
class="stat-label">相似度算法</div></div>
11    </div>
12    <div class="category-section">
13      <h3><img alt="folder icon" data-bbox="225 840 245 855"/> 知识分类</h3>
14      <div class="category-chips">
15        <div v-for="cat in categories" :key="cat.name" class="cat-chip">
16          <span>{{ cat.name }}</span>
17          <span class="cat-count">{{ cat.count }}</span>
18        </div>

```

```

19     </div>
20 </div>
21 <div class="add-dialog" v-if="showAddDialog">
22     <div class="dialog-content">
23         <h3>添加知识</h3>
24         <input v-model="newTitle" placeholder="标题" class="dialog-input" />
25         <textarea v-model="newContent" placeholder="输入知识内容..."
class="dialog-textarea" rows="6"></textarea>
26         <div class="dialog-actions">
27             <button class="btn-cancel" @click="showAddDialog = false">取消
</button>
28             <button class="btn-primary" @click="addKnowledge">确认添加</button>
29         </div>
30     </div>
31 </div>
32 </div>
33 </template>
34
35 <script setup lang="ts">
36 import { ref } from 'vue'
37 const totalChunks = ref(2847)
38 const showAddDialog = ref(false)
39 const newTitle = ref('')
40 const newContent = ref('')
41 const categories = ref([
42     { name: '📰 财经新闻', count: 856 },
43     { name: '📄 年报半年报', count: 1203 },
44     { name: '📖 券商研报', count: 412 },
45     { name: '⚖️ 金融法规', count: 156 },
46     { name: '🔒 内部知识', count: 220 },
47 ])
48 function addKnowledge() { showAddDialog.value = false; newTitle.value = '';
newContent.value = '' }
49 </script>
50
51 <style scoped>
52 .knowledge-view { padding: 24px; height: 100%; overflow-y: auto; }
53 .page-header { display: flex; justify-content: space-between; align-items:
center; margin-bottom: 24px; }
54 .page-header h2 { font-size: 20px; color: #e0e6ed; margin: 0; }
55 .btn-primary { padding: 8px 20px; background: #3182ce; border: none; border-
radius: 8px; color: white; cursor: pointer; }
56 .stats-row { display: flex; gap: 16px; margin-bottom: 24px; }
57 .stat-item { flex: 1; background: rgba(255,255,255,0.04); border: 1px solid
rgba(255,255,255,0.08); border-radius: 12px; padding: 20px; text-align:
center; }
58 .stat-value { font-size: 24px; font-weight: 700; color: #63b3ed; }

```

```
59 .stat-label { font-size: 12px; color: #718096; margin-top: 4px; }
60 .category-section h3 { font-size: 15px; color: #e0e6ed; margin: 0 0 16px; }
61 .category-chips { display: flex; flex-wrap: wrap; gap: 10px; }
62 .cat-chip { display: flex; align-items: center; gap: 8px; padding: 8px 16px;
background: rgba(255,255,255,0.04); border: 1px solid rgba(255,255,255,0.08);
border-radius: 20px; font-size: 13px; color: #a0aec0; }
63 .cat-count { background: rgba(99,179,237,0.2); padding: 2px 8px; border-
radius: 10px; font-size: 12px; color: #63b3ed; }
64 .add-dialog { position: fixed; top: 0; left: 0; right: 0; bottom: 0;
background: rgba(0,0,0,0.7); display: flex; align-items: center; justify-
content: center; z-index: 100; }
65 .dialog-content { background: #1a1f35; border: 1px solid rgba(99,179,237,0.3);
border-radius: 16px; padding: 24px; width: 500px; }
66 .dialog-content h3 { font-size: 18px; color: #e0e6ed; margin: 0 0 16px; }
67 .dialog-input, .dialog-textarea { width: 100%; background:
rgba(255,255,255,0.06); border: 1px solid rgba(99,179,237,0.2); border-radius:
8px; padding: 10px 14px; color: #e0e6ed; font-size: 14px; margin-bottom: 12px;
box-sizing: border-box; font-family: inherit; }
68 .dialog-textarea { resize: vertical; }
69 .dialog-actions { display: flex; justify-content: flex-end; gap: 12px; }
70 .btn-cancel { padding: 8px 20px; background: none; border: 1px solid
rgba(255,255,255,0.2); border-radius: 8px; color: #a0aec0; cursor: pointer; }
71 </style>
```

8. 数据库设计

8.1 ER 关系图 (Mermaid)


```

17     session_id VARCHAR(100) UNIQUE NOT NULL,
18     user_id INTEGER REFERENCES users(id),
19     title VARCHAR(200) DEFAULT '新对话',
20     agent_types JSONB DEFAULT '[]',
21     message_count INTEGER DEFAULT 0,
22     created_at TIMESTAMPTZ DEFAULT NOW(),
23     updated_at TIMESTAMPTZ DEFAULT NOW()
24 );
25 CREATE INDEX idx_session_created ON chat_sessions(session_id, created_at);
26
27 -- 消息记录
28 CREATE TABLE chat_messages (
29     id SERIAL PRIMARY KEY,
30     session_id INTEGER REFERENCES chat_sessions(id),
31     role VARCHAR(20) NOT NULL,
32     content TEXT NOT NULL,
33     agent_type VARCHAR(50),
34     token_count INTEGER DEFAULT 0,
35     metadata JSONB DEFAULT '{}',
36     created_at TIMESTAMPTZ DEFAULT NOW()
37 );
38
39 -- LLM 调用日志 (重点表)
40 CREATE TABLE llm_call_logs (
41     id SERIAL PRIMARY KEY,
42     call_id VARCHAR(100) UNIQUE NOT NULL,
43     session_id VARCHAR(100),
44     user_id INTEGER REFERENCES users(id),
45     model VARCHAR(50) NOT NULL,
46     prompt_tokens INTEGER DEFAULT 0,
47     completion_tokens INTEGER DEFAULT 0,
48     total_tokens INTEGER DEFAULT 0,
49     latency_ms FLOAT DEFAULT 0,
50     cost_usd FLOAT DEFAULT 0,
51     success BOOLEAN DEFAULT true,
52     error_message TEXT,
53     prompt_preview TEXT,
54     response_preview TEXT,
55     metadata JSONB DEFAULT '{}',
56     created_at TIMESTAMPTZ DEFAULT NOW()
57 );
58 CREATE INDEX idx_llm_daily ON llm_call_logs(created_at);
59 CREATE INDEX idx_llm_model_created ON llm_call_logs(model, created_at);
60 CREATE INDEX idx_llm_session ON llm_call_logs(session_id);
61
62 -- 知识文档
63 CREATE TABLE knowledge_documents (

```

```

64     id SERIAL PRIMARY KEY,
65     doc_id VARCHAR(100) UNIQUE NOT NULL,
66     title VARCHAR(500) NOT NULL,
67     category VARCHAR(50) NOT NULL,
68     source VARCHAR(200),
69     content_hash VARCHAR(64),
70     metadata JSONB DEFAULT '{}',
71     chunk_count INTEGER DEFAULT 0,
72     indexed BOOLEAN DEFAULT false,
73     created_at TIMESTAMPTZ DEFAULT NOW(),
74     updated_at TIMESTAMPTZ DEFAULT NOW()
75 );
76
77 -- 文档片段
78 CREATE TABLE document_chunks (
79     id SERIAL PRIMARY KEY,
80     chunk_id VARCHAR(100) UNIQUE NOT NULL,
81     document_id INTEGER REFERENCES knowledge_documents(id),
82     content TEXT NOT NULL,
83     chunk_index INTEGER DEFAULT 0,
84     metadata JSONB DEFAULT '{}',
85     embedding_id VARCHAR(200),
86     created_at TIMESTAMPTZ DEFAULT NOW()
87 );
88 CREATE INDEX idx_chunk_doc ON document_chunks(document_id, chunk_index);
89
90 -- 数据管道
91 CREATE TABLE data_pipelines (
92     id SERIAL PRIMARY KEY,
93     pipeline_name VARCHAR(100) UNIQUE NOT NULL,
94     description TEXT,
95     source_type VARCHAR(50),
96     schedule_cron VARCHAR(100),
97     is_active BOOLEAN DEFAULT true,
98     config JSONB DEFAULT '{}',
99     last_run_at TIMESTAMPTZ
100 );
101
102 -- 管道运行日志
103 CREATE TABLE data_pipeline_run_logs (
104     id SERIAL PRIMARY KEY,
105     run_id VARCHAR(100) UNIQUE NOT NULL,
106     pipeline_name VARCHAR(100) NOT NULL,
107     status VARCHAR(20) NOT NULL,
108     started_at TIMESTAMPTZ DEFAULT NOW(),
109     completed_at TIMESTAMPTZ,
110     records_processed INTEGER DEFAULT 0,

```

```

111     records_failed INTEGER DEFAULT 0,
112     error_message TEXT
113 );
114
115 -- Prompt 版本
116 CREATE TABLE prompt_versions (
117     id SERIAL PRIMARY KEY,
118     version_id VARCHAR(100) UNIQUE NOT NULL,
119     name VARCHAR(100) NOT NULL,
120     agent_type VARCHAR(50),
121     content TEXT NOT NULL,
122     variables JSONB DEFAULT '[]',
123     is_active BOOLEAN DEFAULT false,
124     created_by INTEGER REFERENCES users(id),
125     created_at TIMESTAMPTZ DEFAULT NOW()
126 );
127
128 -- 股票数据缓存
129 CREATE TABLE stock_cache (
130     id SERIAL PRIMARY KEY,
131     stock_code VARCHAR(10) NOT NULL,
132     data_type VARCHAR(50) NOT NULL,
133     data_date VARCHAR(20),
134     data_json JSONB NOT NULL,
135     updated_at TIMESTAMPTZ DEFAULT NOW(),
136     UNIQUE(stock_code, data_type, data_date)
137 );
138 CREATE INDEX idx_stock_cache_ttl ON stock_cache(stock_code, data_type,
    updated_at);

```

9. 部署指南

9.1 Docker Compose 一键部署

代码块

```

1  # 1. 克隆代码
2  git clone https://github.com/your-repo/Financial-AI-Agent.git
3  cd Financial-AI-Agent
4
5  # 2. 配置环境变量
6  cp .env.example .env
7  vim .env # 填写 LLM_API_KEY
8

```

```
9   # 3. 一键启动所有服务
10  docker-compose up -d
11
12  # 4. 检查服务状态
13  docker-compose ps
14
15  # 5. 查看后端日志
16  docker-compose logs -f backend
17
18  # 6. 停止服务
19  docker-compose down
```

9.2 docker-compose.yml 完整配置

代码块

```
1  version: '3.8'
2
3  services:
4    backend:
5      build: ./backend
6      container_name: financial-agent-backend
7      ports: ["8000:8000"]
8      environment:
9        - LLM_API_KEY=${LLM_API_KEY}
10       - LLM_MODEL=${LLM_MODEL:-qwen-plus}
11       - LLM_API_BASE=${LLM_API_BASE}
12       -
13       DATABASE_URL=postgresql+asyncpg://postgres:postgres@postgres:5432/financial_agent
14       - REDIS_HOST=redis
15       - CHROMA_PERSIST_DIR=/app/data/chromadb
16     volumes:
17       - chroma_data:/app/data
18       - ./logs:/app/logs
19     depends_on: [postgres, redis]
20     networks: [financial-net]
21     restart: unless-stopped
22
23   frontend:
24     build: ./frontend
25     container_name: financial-agent-frontend
26     ports: ["3000:80"]
27     depends_on: [backend]
28     networks: [financial-net]
29     restart: unless-stopped
```

```
29
30   postgres:
31     image: timescale/timescaledb:latest-pg15
32     ports: ["5432:5432"]
33     environment:
34       POSTGRES_USER: postgres
35       POSTGRES_PASSWORD: postgres
36       POSTGRES_DB: financial_agent
37     volumes: [postgres_data:/var/lib/postgresql/data]
38     networks: [financial-net]
39     restart: unless-stopped
40
41   redis:
42     image: redis:7-alpine
43     ports: ["6379:6379"]
44     command: redis-server --appendonly yes
45     volumes: [redis_data:/data]
46     networks: [financial-net]
47     restart: unless-stopped
48
49   prometheus:
50     image: prom/prometheus:latest
51     ports: ["9090:9090"]
52     volumes: [./infra/prometheus.yml:/etc/prometheus/prometheus.yml,
prometheus_data:/prometheus]
53     networks: [financial-net]
54     restart: unless-stopped
55
56   grafana:
57     image: grafana/grafana:latest
58     ports: ["3001:3000"]
59     environment: { GF_SECURITY_ADMIN_PASSWORD: admin, GF_USERS_ALLOW_SIGN_UP:
"false" }
60     volumes: [grafana_data:/var/lib/grafana]
61     depends_on: [prometheus]
62     networks: [financial-net]
63     restart: unless-stopped
64
65   networks:
66     financial-net: { driver: bridge }
67
68   volumes:
69     postgres_data:
70     redis_data:
71     chroma_data:
72     prometheus_data:
```

9.3 .env 环境变量配置

代码块

```

1  # ===== LLM 配置 =====
2  LLM_PROVIDER=openai
3  LLM_API_BASE=https://dashscope.aliyuncs.com/compatible-mode/v1
4  LLM_API_KEY=your-dashscope-api-key-here
5  LLM_MODEL=qwen-plus
6
7  # ===== Embedding =====
8  EMBEDDING_API_BASE=https://dashscope.aliyuncs.com/compatible-mode/v1
9  EMBEDDING_API_KEY=your-dashscope-api-key-here
10 EMBEDDING_MODEL=text-embedding-3-large
11 EMBEDDING_DIM=1536
12
13 # ===== 数据库 =====
14 DATABASE_URL=postgresql+asyncpg://postgres:postgres@localhost:5432/financial_agent
15
16 # ===== Redis =====
17 REDIS_HOST=localhost
18 REDIS_PORT=6379
19
20 # ===== 向量库 =====
21 VECTORSTORE_TYPE=chroma
22 CHROMA_PERSIST_DIR=./data/chromadb
23 CHROMA_COLLECTION_NAME=financial_knowledge
24
25 # ===== 安全 =====
26 SECRET_KEY=change-this-to-random-secret-key-in-production

```

9.4 本地开发启动

代码块

```

1  # ===== 后端 =====
2  cd backend
3  python -m venv venv
4  source venv/bin/activate # Windows: venv\Scripts\activate
5  pip install -r requirements.txt
6  cp ../.env.example ../.env
7  uvicorn app.main:app --reload --port 8000

```

```
8
9 # ===== 前端 =====
10 cd frontend
11 npm install
12 npm run dev
13 # 访问 http://localhost:5173
```

9.5 Nginx 生产配置

代码块

```
1  server {
2      listen 80;
3      server_name your-domain.com;
4
5      # 前端静态文件
6      location / {
7          proxy_pass http://frontend:80;
8          proxy_set_header Host $host;
9      }
10
11     # API 反向代理
12     location /api/ {
13         proxy_pass http://backend:8000/api/;
14         proxy_http_version 1.1;
15         proxy_set_header Upgrade $http_upgrade;
16         proxy_set_header Connection "upgrade";
17         proxy_read_timeout 300s;
18     }
19
20     # SSE 流式响应
21     location /api/v1/chat/stream {
22         proxy_pass http://backend:8000/api/v1/chat/stream;
23         proxy_http_version 1.1;
24         proxy_set_header Connection '';
25         proxy_buffering off;
26         proxy_cache off;
27         proxy_read_timeout 300s;
28     }
29 }
```

9.6 Prometheus 告警规则

代码块

```
1 groups:
2   - name: llm_alerts
3     rules:
4       - alert: LLMHighErrorRate
5         expr: rate(llm_calls_total{success="false"}[5m]) > 0.1
6         for: 5m
7         labels: { severity: warning }
8         annotations:
9           summary: "LLM 调用错误率超过 10%"
10
11       - alert: LLMHighLatency
12         expr: histogram_quantile(0.95, rate(llm_latency_seconds_bucket[5m])) >
13           30
14         for: 5m
15         labels: { severity: warning }
16         annotations:
17           summary: "LLM 响应延迟 P95 超过 30 秒"
18
19       - alert: LLMHighCost
20         expr: increase(llm_cost_usd_total[1h]) > 100
21         labels: { severity: critical }
22         annotations:
23           summary: "LLM 调用成本单小时超过 $100"
```